arXiv:2110.11657v3 [cs.CV] 30 Mar 2022

# Projective Manifold Gradient Layer for Deep Rotation Regression

Jiayi Chen[1,2] Yingda Yin[1] Tolga Birdal[3,4] Baoquan Chen[1] Leonidas J. Guibas[3] He Wang[1†]
[1]CFCS, Peking University [2]Beijing Institute for General AI
[3]Stanford University [4]Imperial College London

## Abstract

*Regressing rotations on SO(3) manifold using deep neural networks is an important yet unsolved problem. The gap between the Euclidean network output space and the non-Euclidean SO(3) manifold imposes a severe challenge for neural network learning in both forward and backward passes. While several works have proposed different regression-friendly rotation representations, very few works have been devoted to improving the gradient back-propagating in the backward pass. In this paper, we propose a manifold-aware gradient that directly backpropagates into deep network weights. Leveraging Riemannian optimization to construct a novel projective gradient, our proposed regularized projective manifold gradient (RPMG) method helps networks achieve new state-of-the-art performance in a variety of rotation estimation tasks. Our proposed gradient layer can also be applied to other smooth manifolds such as the unit sphere. Our project page is at https://jychen18.github.io/RPMG.*

## 1. Introduction

Estimating rotations is a crucial problem in visual perception that has broad applications, *e.g.*, in object pose estimation, robot control, camera relocalization, 3D reconstruction and visual odometry [8, 13, 16, 22, 37]. Recently, with the proliferation of deep neural networks, learning to accurately regress rotations is attracting more and more attention. However, the non-Euclidean characteristics of rotation space make accurately regressing rotation very challenging.

As we know, rotations reside in a non-Euclidean manifold, $\mathrm{SO}(3)$ group, whereas the unconstrained outputs of neural networks usually live in Euclidean spaces. This gap between the neural network output space and $\mathrm{SO}(3)$ manifold becomes a major challenge for deep rotation regression, thus tackling this gap becomes an important research topic. One popular research direction is to design learning-friendly rotation representations, *e.g.*, 6D continuous representation from [46] and 10D symmetric matrix representation from [27]. Recently, Levinson *et al.* [25] adopted the vanilla 9D matrix representation discovering that simply replacing the Gram-Schmidt process in the 6D representation [46] with symmetric SVD-based orthogonalization can make this representation superior to the others.

Despite the progress on discovering better rotation representations, the gap between a Euclidean network output space and the non-Euclidean $\mathrm{SO}(3)$ manifold hasn't been completely filled. One important yet long-neglected problem lies in optimization on non-Euclidean manifolds [1]: to optimize on $\mathrm{SO}(3)$ manifold, the optimization variable is a rotation matrix, which contains nine matrix elements; if we naively use *Euclidean gradient*, which simply computes the partial derivatives with respect to each of the nine matrix elements, to update the variable, this optimization step will usually lead to a new matrix off $\mathrm{SO}(3)$ manifold. Unfortunately, we observe that all the existing works on rotation regression simply rely upon *vanilla auto-differentiation* for backpropagation, exactly computing Euclidean gradient and performing such off-manifold updates to predicted rotations. We argue that, for training deep rotation regression networks, the off-manifold components will lead to noise in the gradient of neural network weights, hindering network training and convergence.

To tackle this issue, we draw inspiration from differential geometry, where people leverage *Riemannian optimization* to optimize on the non-Euclidean manifold, which finds the direction of the steepest geodesic path on the manifold and take an on-manifold step. We thus propose to leverage Riemannian optimization and delve deep into the study of the backward pass. Note that this is a fundamental yet currently under-explored avenue, given that most of the existing works focus on a holistic design of rotation regression that is agnostic to forward/backward pass. However, incorporating Riemannian optimization into network training is highly non-trivial and challenging. Although methods of Riemannian optimization allow for optimization on $\mathrm{SO}(3)$ [5, 31], matrix manifolds [1] or general Riemannian manifolds [34, 44], they are not directly applicable to update the weights of the neural networks that are Euclidean. Also,

†: He Wang is the corresponding author (hewang@pku.edu.cn).

approaches like [17] incorporate a Riemannian distance as well as its gradient into network training, however, they do not deal with the *representation* issue.

In this work, we want to *propose a better manifold-aware gradient in the backward pass of rotation regression that directly updates the neural network weights*. We begin by taking a Riemannian optimization step and computing the difference between the rotation prediction and the updated rotation, which is closer to the ground truth. Back-propagating this "error", we encounter the mapping function (or orthogonalization function) that transforms the raw network output to a valid rotation. This projection, which can be the Gram-Schmidt process or SVD orthogonalization [25], is typically a many-to-one mapping. This non-bijectivity provides us with a new design space for our gradient: if we were to use a gradient to update the raw output rotation, many gradients would result in the same update in the final output rotation despite being completely different for backpropagating into the neural network weights. Now the problem becomes: *which gradient is the best for back-propagation when many of them correspond to the same update to the output?*

We observe that this problem is somewhat similar to some problems with ambiguities or multi-ground-truth issues. One example would be the symmetry issue in pose estimation: a symmetric object, *e.g.* a textureless cube, appears the same under many different poses, which needs to be considered when supervising the pose predictions. For supervising the learning in such a problem, Wang *et. al.* [39] proposed to use min-of-N loss [14], which only penalizes the smallest error between the prediction and all the possible ground truths. We therefore propose to find the gradient with the smallest norm that can update the final output rotation to the goal rotation. This *back-projection* process involves finding an element closest to the network output in the inverse image of the goal rotation and projecting the network output to this inverse image space. We therefore coin our gradient *projective manifold gradient*. One thing to note is that this projective gradient tends to shorten the network output, causing the norms of network output to vanish. To fix this problem, we further incorporate a simple regularization into the gradient, leading to our full solution *regularized projective manifold gradient* (RPMG).

Note that our proposed gradient layer operates on the raw network output and can be directly backpropagated into the network weights. Our method is very general and is not tied to a specific rotation representation. It can be coupled with different non-Euclidean rotation representations, including quaternion, 6D representation [46], and 9D rotation matrix representation [25], and can even be used for regressing other non-manifold variables.

We evaluate our devised projective manifold gradient layers on a diverse set of problems involving rotation regression: 3D object pose estimation from 3D point clouds/images, rotation estimation problems without using ground truth rotation supervisions, and please see Appendix E.3 for more experiments on camera relocalization. Our method demonstrates significant and consistent improvements on all these tasks and all different rotation representations tested. Going beyond rotation estimation, we also demonstrate performance improvements on regressing unit vectors (lie on a unit sphere) as an example of an extension to other non-Euclidean manifolds.

We summarize our contribution as below:

- We propose a novel manifold-aware gradient layer, namely *RPMG*, for the backward pass of rotation regression, which can be applied to different rotation representations and losses and used as a "plug-in" at no actual cost.
- Our extensive experiments over different tasks and rotation representations demonstrate the significant improvements from using RPMG.
- Our method can also benefit regression tasks on other manifolds, *e.g.* $\mathcal{S}^2$.

## 2. Related Work

Both rotation parameterization and optimization on SO(3) are well-studied topics. Early deep learning models leverage various rotation representations for pose estimation, *e.g.*, direction cosine matrix (DCM) [19, 43], axis-angle [12, 15, 35], quaternion [11, 21, 23, 42, 45] and Euler-angle [24,30,33]. Recently, [46] points out that Euler-angle, axis-angle, and quaternion are not continuous rotation representations, since their representation spaces are not homeomorphic to SO(3). As better representations for rotation regression, 6D [46], 9D [25], 10D [27] representations are proposed to resolve the discontinuity issue and improve the regression accuracy. A concurrent work [7] examines different manifold mappings theoretically and experimentally, finding out that SVD orthogonalization performs the best when regressing arbitrary rotations. Originating from general Riemannian optimization, [31] presents an easy approach for minimization on the $SO(3)$ group by constructing a local axis-angle parameterization, which is also the tangent space of $SO(3)$ manifold. They backpropagate gradient to the tangent space and use the exponential map to update the current rotation matrix. Most recently, [32] constructs a PyTorch library that supports tangent space gradient backpropagation for 3D transformation groups, (*e.g.*, $SO(3)$, $SE(3)$, $Sim(3)$). This proposed library can be used to implement the Riemannian gradient in our layer.

## 3. Preliminaries

### 3.1. Riemannian Geometry

Following [3, 4], we define an $m$-dimensional *Riemannian manifold* embedded in an ambient Euclidean space

$\mathcal{X} = \mathbb{R}^d$ and endowed with a *Riemannian metric* $\mathbf{G} \triangleq (\mathbf{G}_\mathbf{x})_{\mathbf{x}\in\mathcal{M}}$ to be a smooth curved space $(\mathcal{M}, G)$. A vector $\mathbf{v} \in \mathcal{X}$ is said to be *tangent* to $\mathcal{M}$ at $\mathbf{x}$ iff there exists a smooth curve $\gamma : [0,1] \mapsto \mathcal{M}$ s.t. $\gamma(0) = \mathbf{x}$ and $\dot{\gamma}(0) = \mathbf{v}$. The velocities of all such curves through $\mathbf{x}$ form the *tangent space* $\mathcal{T}_\mathbf{x}\mathcal{M} = \{\dot{\gamma}(0) \,|\, \gamma : \mathbb{R} \mapsto \mathcal{M}$ is smooth around 0 and $\gamma(0) = \mathbf{x}\}$.

**Definition 1** (Riemannian gradient)**.** *For a smooth function $f : \mathcal{M} \mapsto \mathbb{R}$ and $\forall(\mathbf{x}, \mathbf{v}) \in \mathcal{TM}$, we define the* Riemannian gradient *of $f$ as the unique vector field* $\operatorname{grad} f$ *satisfying [6]:*

$$\mathrm{D}f(\mathbf{x})[\mathbf{v}] = \langle \mathbf{v}, \operatorname{grad} f(\mathbf{x})\rangle_\mathbf{x} \tag{1}$$

*where $\mathrm{D}f(\mathbf{x})[\mathbf{v}]$ is the derivation of $f$ by $\mathbf{v}$. It can further be shown (see Appendix B.1) that an expression for* $\operatorname{grad} f$ *can be obtained through the projection of the* Euclidean *gradient orthogonally onto the tangent space*

$$\operatorname{grad} f(\mathbf{x}) = \nabla f(\mathbf{x})_\parallel = \Pi_\mathbf{x}\big(\nabla f(\mathbf{x})\big). \tag{2}$$

*where $\Pi_\mathbf{x} : \mathcal{X} \mapsto \mathcal{T}_\mathbf{x}\mathcal{M} \subseteq \mathcal{X}$ is an orthogonal projector with respect to $\langle\cdot,\cdot\rangle_\mathbf{x}$.*

**Definition 2** (Riemannian optimization)**.** *We consider gradient descent to solve the problems of $\min_{\mathbf{x}\in\mathcal{M}} f(\mathbf{x})$. For a local minimizer or a* stationary point $\mathbf{x}^\star$ *of $f$, the Riemannian gradient vanishes $\operatorname{grad} f(\mathbf{x}^\star) = 0$ enabling a simple algorithm,* Riemannian gradient descent *(RGD):*

$$\mathbf{x}_{k+1} = R_{\mathbf{x}_k}(-\tau_k \operatorname{grad} f(\mathbf{x}_k)) \tag{3}$$

*where $\tau_k$ is the step size at iteration $k$ and $R_{\mathbf{x}_k}$ is the* retraction *usually chosen related to the exponential map.*

### 3.2. Rotation Representations

There are many ways of representing a rotation: classic rotation representations, *e.g.* Euler angles, axis-angle, and quaternion; and recently introduced regression-friendly rotation representations such as *e.g.* 5D [46], 6D [46], 9D [25] and 10D [27] representations. A majority of deep neural networks can output an *unconstrained*, arbitrary $n$-dimensional vector $\mathbf{x}$ in a Euclidean space $\mathcal{X} = \mathbb{R}^n$. For Euler angle and axis-angle representations which use a vector from $\mathbb{R}^3$ to represent a rotation, a neural network can simply output a 3D vector; however, for quaternions, 6D, 9D or 10D representations that lies on non-Euclidean manifolds, manifold mapping function $\pi : \mathbb{R}^n \mapsto \mathcal{M}$ is generally needed for normalization or orthogonalization purposes to convert network outputs to valid elements belonging to the representation manifold. This network Euclidean output space $\mathcal{X}$ is where the representation manifolds reside and therefore are also called ambient space.

**Definition 3** (Rotation representation)**.** *One rotation representation, which lies on a representation manifold $\mathcal{M}$, defines a surjective rotation mapping $\phi : \hat{\mathbf{x}} \in \mathcal{M} \to \phi(\hat{\mathbf{x}}) \in \mathrm{SO}(3)$ and a representation mapping function $\psi : \mathbf{R} \in \mathrm{SO}(3) \to \psi(\mathbf{R}) \in \mathcal{M}$, such that $\phi(\psi) = \mathbf{R} \in \mathrm{SO}(3)$.*

**Definition 4** (Manifold mapping function)**.** *From an ambient space $\mathcal{X}$ to the representation manifold $\mathcal{M}$, we can define a manifold mapping function $\pi : \mathbf{x} \in \mathcal{X} \to \pi(\mathbf{x}) \in \mathcal{M}$, which projects a point $\mathbf{x}$ in the ambient, Euclidean space to a valid element $\hat{\mathbf{x}} = \pi(\mathbf{x})$ on the manifold $\mathcal{M}$.*

We summarize the manifold mappings, the rotation mappings and representation mappings for several non-Euclidean rotation representations below.

**Unit quaternion.** Unit quaternions represent a rotation using a 4D unit vector $\mathbf{q} \in \mathcal{S}^3$ double covering the non-Euclidean 3-sphere *i.e.* $\mathbf{q}$ and $-\mathbf{q}$ identify the same rotation. A network with a final linear activation can only predict $\mathbf{x} \in \mathbb{R}^4$. The corresponding manifold mapping function is usually chosen to be a normalization step, which reads $\pi_q(\mathbf{x}) = \mathbf{x}/\|\mathbf{x}\|$. For rotation and representation mapping, we leverage the standard mappings between rotation and quaternion (see Appendix G).

**6D representation and Gram-Schmidt orthogonalization.** 6D rotation representation [46], lying on Stiefel manifold $\mathcal{V}_2(\mathbb{R}^3)$, uses two orthogonal unit 3D vectors $(\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2)$ to represent a rotation, which are essentially the first two columns of a rotation matrix. Its manifold mapping $\pi_{6D}$ is done through Gram-Schmidt orthogonalization. Its rotation mapping $\phi_{6D}$ is done by adding the third column $\hat{\mathbf{c}}_3 = \hat{\mathbf{c}}_1 \times \hat{\mathbf{c}}_2$. Its representation mapping $\psi_{6D}$ is simply getting rid of the third column $\hat{\mathbf{c}}_3$ from a rotation matrix.

**9D representation and SVD orthogonalization.** To map a raw 9D network output $\mathbf{M}$ to a rotation matrix, [25] use SVD orthogonalization as the manifold mapping function $\pi_{9D}$, as follows: $\pi_{9D}$ first decomposes $\mathbf{M}$ into its left and right singular vectors $\{\mathbf{U}, \mathbf{V}^\top\}$ and singular values $\Sigma$, $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^\top$; then it replaces $\Sigma$ with $\Sigma' = \mathrm{diag}(1, 1, \det(\mathbf{U}\mathbf{V}^\top))$ and finally, computes $\mathbf{R} = \mathbf{U}\Sigma'\mathbf{V}^\top$ to get the corresponding rotation matrix $\mathbf{R} \in \mathrm{SO}(3)$. As this representation manifold is $\mathrm{SO}(3)$, both the rotation and representation mapping functions are simply identity.

**10D representation.** [27] propose a novel 10D representation for rotation matrix. The manifold mapping function $\pi_{10D}$ maps $\boldsymbol{\theta} \in \mathbb{R}^{10}$ to $\mathbf{q} \in \mathcal{S}^3$ by computing the eigenvector corresponding to the smallest eigenvalue of $\mathbf{A}(\boldsymbol{\theta})$, expressed as $\pi_{10D}(\mathbf{x}) = \min_{\mathbf{q}\in\mathcal{S}^3} \mathbf{q}^\top\mathbf{A}(\mathbf{x})\mathbf{q}$, in which

$$\mathbf{A}(\boldsymbol{\theta}) = \begin{bmatrix} \theta_1 & \theta_2 & \theta_3 & \theta_4 \\ \theta_2 & \theta_5 & \theta_6 & \theta_7 \\ \theta_3 & \theta_6 & \theta_8 & \theta_9 \\ \theta_4 & \theta_7 & \theta_9 & \theta_{10} \end{bmatrix}. \tag{4}$$

Since the representation manifold is $\mathcal{S}^3$, the rotation and representation mapping are the same as unit quaternion.

### 3.3. Deep Rotation Regression

We conclude this section by describing the ordinary forward and backward passes of a neural network based rota-

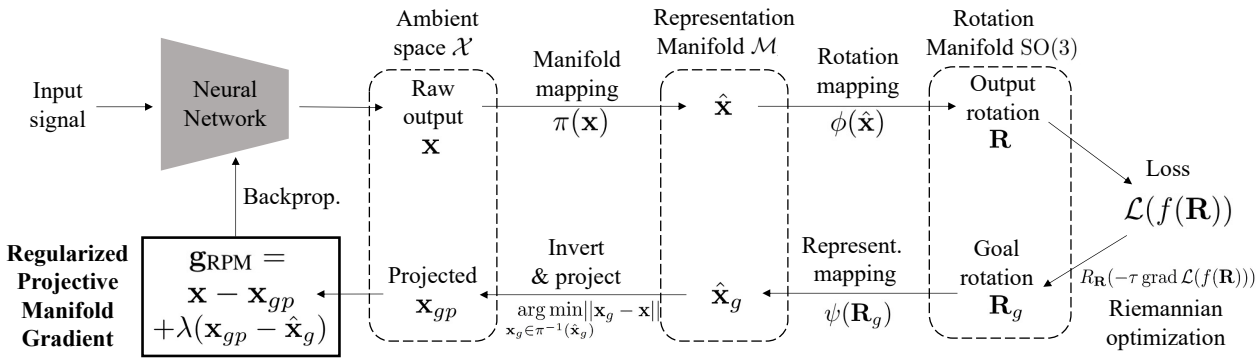


Figure 1. **Projective Manifold Gradient Layer.** In the forward pass, the network predicts a raw output $\mathbf{x}$, which is then transformed into a valid rotation $\mathbf{R} = \phi(\pi(\mathbf{x}))$. We leave this forward pass unchanged and only modify the backward pass. In the backward pass, we first use Riemannian optimization to get a goal rotation $\mathbf{R}_g$ and map it back to $\hat{\mathbf{x}}_g$ on the representation manifold $\mathcal{M}$. After that we find the element $\mathbf{x}_{gp}$ which is closest to the raw output in the inverse image of $\hat{\mathbf{x}}_g$, and finally get the gradient $\mathbf{g}_{\mathbf{RPM}}$ we want.

tion regression, as used in [25, 46].

**Forward and backward passes.** Assume, for a rotation representation, the network predicts $\mathbf{x} \in \mathcal{X}$, then the manifold mapping $\pi$ will map $\mathbf{x}$ to $\hat{\mathbf{x}} = \pi(\mathbf{x}) \in \mathcal{M}$, followed by a rotation mapping $\phi$ that finally yields the output rotation $\mathbf{R} = \phi(\hat{\mathbf{x}}) = \phi(\pi(\mathbf{x}))$. Our work only tackles the backward pass and keeps the forward pass unchanged, as shown in the top part of Figure 1. The gradient in the backward-pass is simply computed using Pytorch autograd method, that is $\mathbf{g} = f'(\mathbf{R})\phi'(\hat{\mathbf{x}})\pi'(\mathbf{x})$.

**Loss function.** The most common choice for supervising rotation matrix is L2 loss, $\|\mathbf{R} - \mathbf{R}_{gt}\|_F^2$ , as used by [25, 46]. This loss is equal to $4 - 4\cos(< \mathbf{R}, \mathbf{R}_{gt} >)$, where $< \mathbf{R}, \mathbf{R}_{gt} >$ represents the angle between $\mathbf{R}$ and $\mathbf{R}_{gt}$.

## 4. Method

**Overview.** In this work, we propose a *projective manifold gradient layer*, without changing the forward pass of a given rotation regressing network, as shown in Figure 1. Our focus is to find a better gradient $\mathbf{g}$ of the loss function $\mathcal{L}$ with respect to the network raw output $\mathbf{x}$ for backpropagation into the network weights.

Let's start with examining the gradient of network output $\mathbf{x}$ in a general case – regression in Euclidean space. Given a ground truth $\mathbf{x}_{gt}$ and the L2 loss $\|\mathbf{x} - \mathbf{x}_{gt}\|^2$ that maximizes the likelihood in the presence of Gaussian noise in $\mathbf{x}$, the gradient would be $\mathbf{g} = 2(\mathbf{x} - \mathbf{x}_{gt})$.

In the case of rotation regression, we therefore propose to find a proper $\mathbf{x}^* \in \mathcal{X}$ for a given ground truth $\mathbf{R}_{gt}$ or a computed goal rotation $\mathbf{R}_g$ when the ground truth rotation is not available, and then simply use $\mathbf{x} - \mathbf{x}^*$ as our gradient to backpropagate into the network.

Note that finding such an $\mathbf{x}^*$ can be challenging. Assuming we know $\mathbf{R}_{gt}$, finding an $\mathbf{x}^*$ involves inverting $\phi$ and $\pi$ since the network output $\mathbf{R} = \phi(\pi(\mathbf{x}))$. Furthermore, we may not know $\mathbf{R}_{gt}$ under indirect rotation supervision (*e.g.*, flow loss as used in PoseCNN [42]) and self-supervised rotation estimation cases (*e.g.*, 2D mask loss as used in [38]).

In this work, we introduce the following techniques to mitigate these problems: (i) we first take a Riemannian gradient to compute a goal rotation $\mathbf{R}_g \in \mathrm{SO}(3)$, which does not rely on knowing $\mathbf{R}_{gt}$, as explained in Section 4.1; (ii) we then find the set of all possible $\mathbf{x}_g$s that can be mapped to $\mathbf{R}_g$, or in other words, the inverse image of $\mathbf{R}_g$ under $\pi$ and $\phi$; (iii) we find $\mathbf{x}_{gp}$ which is the element in this set closest to $\mathbf{x}$ in the Euclidean metric and set it as "$\mathbf{x}^*$". We will construct our gradient using this $\mathbf{x}^*$, as explained in 4.2. (iv) we add a regularization term to this gradient forming $\mathbf{g}_{RPMG}$ as explained in 4.3. The whole backward pass leveraging our proposed regularized projective manifold gradient is shown in the lower half of Figure 1.

### 4.1. Riemannian Gradient and Goal Rotation

To handle rotation estimation with/without direct rotation supervision, we first propose to compute the Riemannian gradient of the loss function $\mathcal{L}$ with respect to the output rotation $\mathbf{R}$ and find a goal rotation $\mathbf{R}_g$ that is presumably closer to the ground truth rotation than $\mathbf{R}$.

Assume the loss function is in the following form $\mathcal{L}(f(\mathbf{R}))$, where $\mathbf{R} = \pi(\phi(\mathbf{x}))$ is the output rotation and $f$ constructs a loss function that compares $\mathbf{R}$ to the ground truth rotation $\mathbf{R}_{gt}$ directly or indirectly. Given $\mathbf{R}(\mathbf{x})$ and $\mathcal{L}(f(\mathbf{R}(\mathbf{x})))$, we can perform one step of Riemannian optimization yielding our goal rotation $\mathbf{R}_g \leftarrow R_{\mathbf{R}}(-\tau \operatorname{grad} \mathcal{L}(f(\mathbf{R})))$, where $\tau$ is the step size of Riemannian gradient and can be set to a constant as a hyperparameter or varying during the training. For L2 loss $\|\mathbf{R} - \mathbf{R}_{gt}\|_F^2$, the Riemannian gradient is always along the geodesic path between $\mathbf{R}$ and $\mathbf{R}_{gt}$ on SO(3) [20]. In this case, $\mathbf{R}_g$ can generally be seen as an intermediate goal between $\mathbf{R}$ and $\mathbf{R}_{gt}$ dependent on $\tau$. Gradually increasing $\tau$ from 0 will first make $\mathbf{R}_g$ approach $\mathbf{R}_{gt}$ starting with $\mathbf{R}_g = \mathbf{R}$, and then reach $\mathbf{R}_{gt}$ where we denote $\tau = \tau_{gt}$, and

finally going beyond $\mathbf{R}_{gt}$. Although, when $\mathbf{R}_{gt}$ is available, one can simply set $\mathbf{R}_g = \mathbf{R}_{gt}$, we argue that this is just a special case under $\tau = \tau_{gt}$. For scenarios where $\mathbf{R}_{gt}$ is unavailable, *e.g.*, in self-supervised learning cases (see in Section 5.3), we don't know $\mathbf{R}_{gt}$ and $\tau_{gt}$, thus we need to compute $\mathbf{R}_g$ using Riemannian optimization. In the sequel, we only use $\mathbf{R}_g$ for explaining our methods without loss of generality. See Section 4.3 for how to choose $\tau$.

### 4.2. Projective Manifold Gradient

Given $\mathbf{R}_g$, we can use the representation mapping $\psi$ to find the corresponding $\hat{\mathbf{x}}_g = \psi(\mathbf{R}_g)$ on the representation manifold $\mathcal{M}$. However, further inverting $\pi$ and finding the corresponding $\mathbf{x}_g \in \mathcal{X}$ is a non-trivial problem, due to the projective nature of $\pi$. In fact, there are many $\mathbf{x}_g$s that satisfy $\pi(\mathbf{x}_g) = \hat{\mathbf{x}}_g$. It seems that we can construct a gradient $\mathbf{g} = (\mathbf{x} - \mathbf{x}_g)$ using any $\mathbf{x}_g$ that satisfies $\pi(\mathbf{x}_g) = \hat{\mathbf{x}}_g$. No matter which $\mathbf{x}_g$ we choose, if this gradient were to update $\mathbf{x}$, it will result in the same $\mathbf{R}_g$. But, when backpropagating into the network, those gradients will update the network weights differently, potentially resulting in different learning efficiency and network performance.

Formally, we formulate this problem as *a multi-ground-truth problem* for $\mathbf{x}$: we need to find the best $\mathbf{x}^*$ to supervise from the inverse image of $\hat{\mathbf{x}}_g$ under the mapping $\pi$. We note that similar problems have been seen in pose supervision dealing with symmetry as in [39], where one needs to find one pose to supervise when there are many poses under which the object appears the same. [39] proposed to use a min-of-N strategy introduced by [14]: from all possible poses, taking the pose that is closest to the network prediction as ground truth. A similar strategy is also seen in supervising quaternion regression, as $\mathbf{q}$ and $-\mathbf{q}$ stand for the same rotation. One common choice of the loss function is therefore $\min\{\mathcal{L}(\mathbf{q}, \mathbf{q}_{gt}), \mathcal{L}(\mathbf{q}, -\mathbf{q}_{gt})\}$ [27], which penalizes the distance to the closest ground truth quaternion.

Inspired by these works, we propose to choose our gradient among all the possible gradients with the lowest level of redundancy, *i.e.*, we require $\mathbf{x}^*$ to be the one closest to $\mathbf{x}$, or in other words, the gradient to have the smallest norm, meaning that we need to find the projection point $\mathbf{x}_{gp}$ of $\mathbf{x}$ to all the valid $\mathbf{x}_g$:

$$\mathbf{x}_{gp} = \underset{\pi(\mathbf{x}_g)=\hat{\mathbf{x}}_g}{\operatorname{argmin}} \|\mathbf{x} - \mathbf{x}_g\|_2 \tag{5}$$

We then can construct our *projective manifold gradient* (PMG) as $\mathbf{g}_{PM} = \mathbf{x} - \mathbf{x}_{gp}$. We will denote the naive gradient $\mathbf{g}_M = \mathbf{x} - \hat{\mathbf{x}}_g$ as *manifold gradient* (MG).

Here we provide another perspective on why a network may prefer PMG. In the case where a deep network is trained using stochastic gradient descent (SGD), the final gradient used to update the network weights is averaged across the gradients of all the batch instances. If gradients

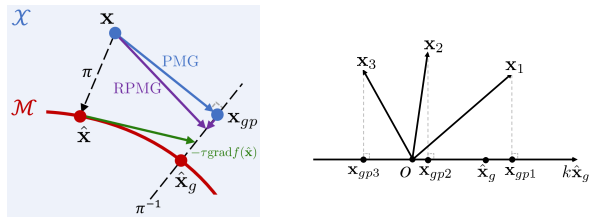


Figure 2. **Illustration for regularized projective manifold gradient. Left**: In the forward pass, we simply project $\mathbf{x}$ to $\hat{\mathbf{x}}$ by $\pi$. In the backward pass, first we compute a Riemannian gradient, which is shown as the *green* arrow. After getting a next goal $\hat{\mathbf{x}}_g \in \mathcal{M}$ by Riemannian optimization, we find the inverse projection $\mathbf{x}_{gp}$ of $\hat{\mathbf{x}}_g$, which leads to our *projective manifold gradient*, shown as the *blue* arrow. With a regularization term, we can get our final *regularized projective manifold gradient*, as the *purple* arrow. **Right**: Projection point $\hat{\mathbf{x}}_{gp}$ in the case of quaternion.

from different batch instances contain different levels of redundancy, then the averaged gradient may be biased or not even appropriate. This argument is generally applicable to all stochastic optimizers (*e.g.*, Adam [2])

**Inverting $\pi$.** There are many ways to solve this projection problem for different manifold mapping functions $\pi$. For example, we can formulate this as a constrained optimization problem. For the manifold mapping functions we consider, we propose the following approach: we first solve for the inverse image $\pi^{-1}(\hat{\mathbf{x}}_g)$ of $\hat{\mathbf{x}}_g$ in the ambient space $\mathcal{X}$ analytically, which reads $\pi^{-1}(\hat{\mathbf{x}}_g) = \{\mathbf{x}_g \in \mathcal{X} \mid \pi(\mathbf{x}_g) = \hat{\mathbf{x}}_g\}$; we then project $\mathbf{x}$ onto this inverse image space. Note that, sometimes only a superset of this inverse image can be found analytically, requiring certain constraints on $\mathbf{x}_{gp}$ to be enforced.

Here we list the inverse image $\pi^{-1}(\hat{\mathbf{x}}_g)$ and the projection point $\mathbf{x}_{gp}$ for different rotation representations and their corresponding manifold mapping $\pi$. Please refer to Appendix B.2 for detailed derivations.

**Quaternion.** With $\pi_q(\mathbf{x}) = \mathbf{x}/\|\mathbf{x}\|$, $\mathbf{x} \in \mathbb{R}^4$, and $\hat{\mathbf{x}}_g \in \mathcal{S}^3$: $\pi_q^{-1}(\hat{\mathbf{x}}_g) = \{\mathbf{x} \mid \mathbf{x} = k\hat{\mathbf{x}}_g, k \in \mathbb{R} \text{ and } k > 0\}$, which is a ray in the direction of $\hat{\mathbf{x}}_g$ starting from the origin. Without considering the constraint of $k > 0$ , an analytical solution to this projection point $\mathbf{x}_{gp}$ of $\mathbf{x}$ onto this line can be derived: $\mathbf{x}_{gp} = (\mathbf{x} \cdot \hat{\mathbf{x}}_g)\hat{\mathbf{x}}_g$.

**6D representation.** With $\pi_{6D}$ as Gram-Schmidt process, $\mathbf{x} = [\mathbf{u}, \mathbf{v}] \in \mathbb{R}^6$, and $\hat{\mathbf{x}}_g \in \mathcal{V}_2(\mathbb{R}^3)$: $\pi_{6D}^{-1}(\hat{\mathbf{x}}_g) = \{[k_1\hat{\mathbf{u}}_g, k_2\hat{\mathbf{u}}_g + k_3\hat{\mathbf{v}}_g] \mid k_1, k_2, k_3 \in \mathbb{R} \text{ and } k_1, k_3 > 0\}$ (the former is a ray whereas the latter spans a half plane). Without considering the constraint of $k_1, k_3 > 0$, the projection point $\mathbf{x}_{gp}$ can be analytically represented as $\mathbf{x}_{gp} = [(\mathbf{u} \cdot \hat{\mathbf{u}}_g)\hat{\mathbf{u}}_g, (\mathbf{v} \cdot \hat{\mathbf{u}}_g)\hat{\mathbf{u}}_g + (\mathbf{v} \cdot \hat{\mathbf{v}}_g)\hat{\mathbf{v}}_g]$

**9D representation.** With $\pi_{9D}(\mathbf{x})$ as SVD orthogonalization, $\mathbf{x} \in \mathbb{R}^{3\times3}$, and $\hat{\mathbf{x}}_g \in$ SO(3), the analytical expression for $\pi_{9D}^{-1}$ is available when we ignore the positive singular value constraints, which gives $\pi_{9D}^{-1}(\hat{\mathbf{x}}_g) = \{\mathbf{S}\hat{\mathbf{x}}_g \mid \mathbf{S} =$

$\mathbf{S}^\top\}$. We can further solve the projection point $\mathbf{x}_{gp}$ with an elegant representation $\mathbf{x}_{gp} = \frac{\mathbf{x}\hat{\mathbf{x}}_g^T + \hat{\mathbf{x}}_g\mathbf{x}^T}{2}$.

**10D representation.** Please refer to Appendix B.2 for the derivation and expression of $\mathbf{x}_{gp}$.

### 4.3. Regularized Projective Manifold Gradient

**Issues in naive projective manifold gradient.** In the right plot of Figure 2, we illustrate this projection process for several occasions where $\mathbf{x}$ takes different positions relative to $\mathbf{x}_g$. We demonstrate that there are two issues in this process.

First, no matter where $\mathbf{x}$ is in, the projection operation will shorten the length of our prediction because $\|\mathbf{x}_{gp}\| < \|\mathbf{x}\|$ is always true for all of 4D/6D/9D/10D representation. This will cause the length norm of the prediction of the network to become very small as the training progresses (see Figure 3). The shrinking network output will keep increasing the effective learning rate, preventing the network from convergence and leading to great harm to the network performance (see Table 2 and Figure 3 for ablation study).

Second, when the angle between $\mathbf{x}$ and $\hat{\mathbf{x}}_g$ becomes larger than $\pi/2$ (in the case of $\mathbf{x} = \mathbf{x}_3$), the naive projection $\mathbf{x}_{gp}$ will be in the opposite direction of $\hat{\mathbf{x}}_g$ and can not be mapped back to $\hat{\mathbf{x}}_g$ under $\pi_q$, resulting in a wrong gradient. The same set of issues also happens to 6D, 9D and 10D representations. The formal reason is that the analytical solution of the inverse image assumes certain constraints are satisfied, which is usually true only when either $\hat{\mathbf{x}}_g$ is not far from $\mathbf{x}$ or the network is about to converge.

**Regularized projective manifold gradient.** To solve the first issue, we propose to add a regularization term $\mathbf{x}_{gp} - \hat{\mathbf{x}}_g$ to the projective manifold gradient, which can avoid the length vanishing problem. The *regularized projective manifold gradient* then reads:

$$\mathbf{g}_{RPM} = \mathbf{x} - \mathbf{x}_{gp} + \lambda(\mathbf{x}_{gp} - \hat{\mathbf{x}}_g), \tag{6}$$

where $\lambda$ is a regularization coefficient. See the left plot of Figure 2 for an illustration.

**Discussion on the hyperparameters $\lambda$ and $\tau$.** Our method apparently introduces two additional hyperparameters, $\lambda$ and $\tau$, however, we argue that this doesn't increase the searching space of hyperparameters for our method.

For $\lambda$, the only requirement is that $\lambda$ is small (we simply set to 0.01), because: (1) we want the projective manifold gradient $(\mathbf{x} - \mathbf{x}_{gp})$ to be the major component of our gradient; (2) since this regularization is roughly proportional to the difference in prediction length and a constant, a small lambda is enough to prevent the length from vanishing and, in the end, the prediction length will stay roughly constant at the equilibrium under projection and regularization. In the ablation study of Section 5.1, we show that the performance is robust to the change of $\lambda$. Note that, on the other extreme, when $\lambda = 1$, $\mathbf{g}_{RPM}$ becomes $\mathbf{g}_M$.

For $\tau$, we propose a ramping up schedule which is well-motivated. To tackle the second problem of reversed gradient, we need a small $\tau_{init}$ to keep $\mathbf{R}_g$ close to $\mathbf{R}$ at the beginning of training. But when the network is about to converge, we will prefer a $\tau_{converge}$ which can keep $\mathbf{R}_g$ close to $\mathbf{R}_{gt}$ for better convergence. We cannot directly set $\tau_{converge}$ to $\tau_{\text{gt}}$, which is introduced in 4.1, because $\tau_{\text{gt}}$ is not a constant and cannot be used in Riemannian Optimization. However, if we want to tackle the problem of reversed gradient, we must need Riemannian Optimization and $\tau_{init}$. Thus we need a constant approximation of $\tau_{\text{gt}}$ when the angle between $\mathbf{R}$ and $\mathbf{R}_{gt}$ converges to 0. Note that $\tau_{converge}$ can be derived analytically when the loss function is the most widely used L2 loss or geodesic loss(please refer to Appendix B.1 for details), and therefore doesn't need to be tuned. Therefore we propose to increase $\tau$ from a small value $\tau_{init}$, leading to a slow warm-up and, as the training progresses, we gradually increase it to the final $\tau = \tau_{converge}$ by ten uniform steps. This strategy further improves our performance.

## 5. Experiments

We investigate popular rotation representations and find our methods greatly improve the performance in different kinds of tasks. For our regularized projective manifold gradient (**RPMG**), we apply it in the backpropagation process of Quaternion, 6D, 9D and 10D, without changing the forward pass, leading to three new methods **RPMG-Quat**, **RPMG-6D**, **RPMG-9D** and **RPMG-10D**. We compare the following seven baselines: **Euler angle**, **axis-angle**, **Quaternion**, **6D** [46], **9D** [25], **9D-Inf** [25] and **10D** [27]. We adopt three evaluation metrics: mean, median, and 5° accuracy of (geodesic) errors between predicted rotation and ground truth rotation. For most of our experiments, we set the regularization term $\lambda = 0.01$ and increase $\tau$ from $\tau_{init} = 0.05$ to $\tau_{converge} = 0.25$ by ten uniform steps. We further show and discuss the influence of different choices of these two hyperparameters in our ablation studies.

### 5.1. 3D Object Pose Estimation from Point Clouds

**Experimental setting.** As in [9], we use the complete point clouds generated from the models in ModelNet-40 [40]. We use the same train/test split as in [9] and report the results of *airplane*, *chair*, *sofa*, *toilet* and *bed* those five categories because they exhibit less rotational symmetries. Given one shape point clouds of a specific category, the network learns to predict the 3D rotation of the input point clouds from the predefined canonical view of this category [39]. We replace the point clouds alignment task used in [25, 46] (which has almost been solved) by this experiment since it is more challenging and closer to real-world applications (no canonical point clouds is given to the network).

We use a PointNet++ [28] network as our backbone, supervised by L2 loss between the predicted rotation matrix

| Methods | Airplane | | | Chair | | | Sofa | | | Toilet | | | Bed | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ |
| Euler | 125 | 131 | 0 | 13.6 | 9.0 | 17 | 120 | 125 | 0 | 127 | 133 | 0 | 113 | 122 | 0 |
| Axis-Angle | 10.8 | 8.2 | 22 | 16.4 | 10.9 | 9 | 24.1 | 14.6 | 6 | 21.9 | 13.0 | 9 | 25.5 | 11.0 | 16 |
| Quaternion | 9.7 | 7.6 | 27 | 16.7 | 11.4 | 12 | 20.4 | 12.7 | 10 | 16.0 | 9.3 | 17 | 27.8 | 11.3 | 14 |
| 6D | 5.5 | 4.7 | 54 | 9.8 | 6.4 | 35 | 14.6 | 9.5 | 15 | 9.3 | 6.8 | 33 | 24.7 | 9.6 | 17 |
| 9D | 4.7 | 3.9 | 67 | 7.9 | 5.4 | 44 | 15.7 | 10.0 | 14 | 10.3 | 6.9 | 30 | 22.3 | 8.5 | 20 |
| 9D-Inf (MG-9D) | 3.1 | 2.5 | 90 | 5.3 | 3.7 | 69 | 7.8 | 5.0 | 50 | 4.2 | 3.3 | 75 | 12.9 | 4.6 | 55 |
| 10D | 5.3 | 4.2 | 61 | 8.9 | 6.0 | 38 | 15.1 | 10.3 | 13 | 10.7 | 6.5 | 35 | 23.1 | 8.7 | 19 |
| RPMG-Quat | 3.2 | 2.4 | 88 | 6.3 | 3.7 | 67 | 8.1 | 4.5 | 57 | 4.9 | 3.5 | 74 | 13.3 | 3.6 | 70 |
| RPMG-6D | 2.6 | 2.1 | **94** | **5.0** | **3.1** | 74 | 6.6 | 3.6 | 70 | **3.8** | 2.9 | **83** | 13.5 | 2.7 | 81 |
| RPMG-9D | **2.5** | **2.0** | **94** | 5.1 | **3.1** | **76** | **6.1** | **3.1** | **77** | 4.3 | **2.7** | **83** | **10.9** | **2.5** | **86** |
| RPMG-10D | 2.8 | 2.2 | 93 | 5.1 | 3.2 | 75 | 6.5 | 3.2 | 72 | 4.9 | 2.8 | 82 | 13.5 | 2.7 | 82 |

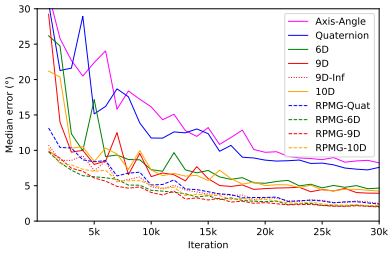


Table 1. **Pose estimation from ModelNet40 point clouds.** Left: a comparison of methods by mean, median, and 5° accuracy of (geodesic) errors after 30k training steps. Mn, Md and Acc are abbreviations of mean, median and 5° accuracy. Right: median test error of *airplane* in different iterations during training.

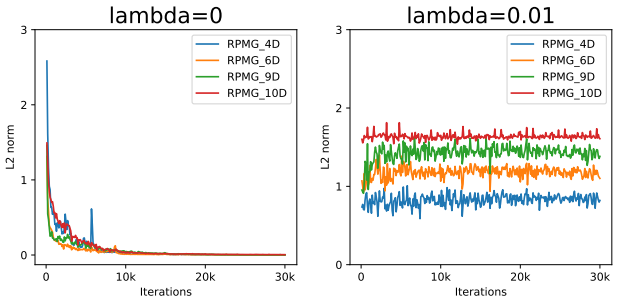


Figure 3. **Average L2 norm of the network raw output** $\mathbf{x}$ during training. Left: PMG-4D/6D/9D/10D (w/o reg. $\lambda = 0$). Right: RPMG-4D/6D/9D/10D (w/ reg. $\lambda = 0.01$)

$\mathbf{R}$ and the ground truth rotation matrix $\mathbf{R}_{gt}$. To facilitate a fair comparison between multiple methods, we use the same set of hyperparameters in all the experiments. Please see Appendix F for more details.

**Analysis of results.** The results are shown in Table 1. We see a great improvement of our methods in all three rotation representations. In this experiment, one may find **9D-Inf** also leads to a good performance, which is actually a special case of our method with $\lambda = 1$, or in other words, it is MG with $\tau = \tau_{gt}$. Nonetheless, in Table 3, we can observe a larger gap. Also, this simple loss may lead to bad performance when $\mathbf{R}_{gt}$ is unavailable in Section 5.3.

**Ablation study on $\lambda$.** As mentioned in Section 4.3, naively using **PMG** without any regularization, corresponding to setting $\lambda = 0$, will lead to length vanishing; To maintain the length of prediction roughly constant, we only need to add a small $\lambda$. In Figure 3, We show the length vanishing problem without regularization and stabilized length with a small regularization. In Table 1, we show that the network performs much better when we have a small $\lambda$ (**RPMG**) than $\lambda = 0$ (**PMG**) or $\lambda = 1$ (**MG**), which deviates too far away from the desired projective manifold gradient. As for the exact value of $\lambda$, our experiments show that our method is robust to the choice of $\lambda$ as long as it is small. Table 2 also shows that $\lambda = 0.01, 0.005, 0.05$ all lead to similar performance, thus freeing us from tuning the parameter $\lambda$.

**Ablation study on $\tau$.** For the choices of $\tau$, Table 2 shows that our proposed strategy, which ramps up $\tau$ from a small $\tau_{\text{init}}$ to $\tau_{\text{converge}}$, works the best. The reason is that: a big $\tau$, when training begins, may cause the problem of reversed gradient discussed in Section 4.3. On the other side, a small $\tau$ at the end of training will slow down the training process and can do harm to convergence. Note that, the performance is not very sensitive to the exact value, which means we don't require a parameter tuning for $\tau$ even in general cases. We are good even with simply setting $\tau = \tau_{gt}$.

| Methods | | | Mean (°)↓ | Med (°)↓ | 5°Acc (%)↑ |
|---|---|---|---|---|---|
| L2 6D | - | - | 5.50 | 4.67 | 54.4 |
| MG-6D | $\lambda = 1$ | $\tau_{converge}$ | 3.51 | 2.95 | 85.2 |
| | | $\tau_{gt}$ | 3.19 | 2.72 | 87.8 |
| PMG-6D | $\lambda = 0$ | $\tau_{converge}$ | 57.65 | 45.22 | 0.2 |
| | | $\tau_{gt}$ | 133 | 136 | 0.0 |
| RPMG-6D | $\lambda = 0.01$ | $\tau_{init}$ | 2.67 | 2.18 | 93.1 |
| | | $\tau_{converge}$ | 2.71 | 2.14 | 93.2 |
| | | $\tau_{\text{gt}}$ | 3.02 | 2.14 | 89.5 |
| | | $\tau_{init} \rightarrow \tau_{converge}$ | 2.59 | 2.07 | 93.6 |
| | $\lambda = 0.05$ | $\tau_{init} \rightarrow \tau_{converge}$ | 2.73 | 2.23 | 92.9 |
| | $\lambda = 0.005$ | | **2.52** | **2.05** | **94.3** |

Table 2. **Ablation study of pose estimation from *airplane* point clouds.** Here MG stands for manifold gradient $\mathbf{x} - \hat{\mathbf{x}}_g$, corresponding to set $\lambda = 1$; PMG stands for projective manifold gradient $\mathbf{x} - \mathbf{x}_{gp}$, corresponding to set $\lambda = 0$.

## 5.2. 3D Rotation Estimation from ModelNet Images

In this experiment, we follow the setting in [25] to estimate poses from 2D images. Images are rendered from ModelNet-10 [40] objects from arbitrary viewpoints [26]. A MobileNet [18] is used to extract image features and three MLPs to regress rotations. We use the same categories as in Experiment 5.1 except *airplane*, since ModelNet-10 doesn't have this category. We didn't quote the numbers from [25] since we conduct all the experiments using the same set of hyperparameters to ensure a fair comparison. Please see Appendix F.2 for more details.

The results are shown in Table 3. Our RPMG layer boosts the performance of all three representations significantly. See the curves with the same color for comparison.

| Methods | Chair | | | Sofa | | | Toilet | | | Bed | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ | Mn↓ | Md↓ | Acc↑ |
| Euler | 21.5 | 10.9 | 10 | 27.5 | 12.0 | 9 | 14.9 | 8.5 | 19 | 27.6 | 9.6 | 17 |
| Axis-Angle | 25.7 | 14.3 | 7 | 30.3 | 14.6 | 6 | 20.3 | 13.0 | 8 | 36.3 | 16.7 | 4 |
| Quaternion | 25.8 | 15.0 | 6 | 30.0 | 15.7 | 6 | 20.6 | 13.0 | 8 | 34.1 | 15.5 | 5 |
| 6D | 19.6 | 9.1 | 19 | 17.5 | 7.3 | 27 | 10.9 | 6.2 | 37 | 32.3 | 11.7 | 11 |
| 9D | 17.5 | 8.3 | 23 | 19.8 | 7.6 | 25 | 11.8 | 6.5 | 34 | 30.4 | 11.1 | 13 |
| 9D-Inf | 12.1 | 5.1 | 49 | 12.5 | 3.5 | 70 | 7.6 | 3.7 | 67 | 22.5 | 4.5 | 56 |
| 10D | 18.4 | 9.0 | 20 | 20.9 | 8.7 | 20 | 11.5 | 5.9 | 39 | 29.9 | 11.5 | 11 |
| RPMG-Quat | 13.0 | 5.9 | 40 | 13.0 | 3.6 | 67 | 8.6 | 4.2 | 61 | 23.2 | 4.9 | 51 |
| RPMG-6D | 12.9 | 4.7 | 53 | 11.5 | 2.8 | 77 | 7.8 | 3.4 | 71 | 20.3 | 3.6 | 67 |
| RPMG-9D | **11.9** | **4.4** | **58** | **10.5** | **2.4** | **82** | 7.5 | 3.2 | 75 | 20.0 | **2.9** | **76** |
| RPMG-10D | 12.8 | 4.5 | 55 | 11.2 | **2.4** | **82** | **7.2** | **3.0** | **76** | **19.2** | **2.9** | 75 |

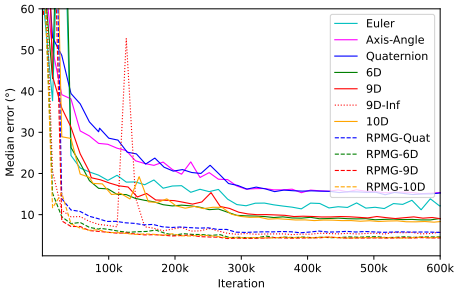


Table 3. **Pose estimation from ModelNet10 images.** Left: a comparison of methods by mean(°), median(°), and 5° accuracy(%) of (geodesic) errors after 600k training steps. Mn, Md and Acc are abbreviations of mean, median and 5° accuracy. Right: median test error of *chair* in different iterations during training.

## 5.3. Rotation Estimation without Supervision

**Self-supervised instance-level rotation estimation from point clouds.** For one complete chair instance $Z$, given a complete observation $X$, we estimate its pose $\mathbf{R}$. We then use Chamfer distance between $Z$ and $\mathbf{R}^{-1}X$ as a self-supervised loss. The network structure and training settings are all the same as Experiment 5.1, except here we use $\tau = 2$. See Appendix E.2 for how to find a suitable $\tau$.

The interesting thing here is that vanilla **9D-Inf** fails while our methods still perform very well. We think that this is because the Chamfer distance loss will greatly enlarge the effect of the noisy part (which is introduced by $\lambda$) in gradient, leading to a very bad performance.

| Methods | Mean (°)↓ | Med (°)↓ | 3°Acc (%)↑ |
|---|---|---|---|
| Euler | 131.9 | 139.1 | 0.0 |
| Axis-Angle | 4.5 | 3.8 | 34.5 |
| Quaternion | 4.3 | 3.5 | 37.5 |
| 6D | 55.1 | 6.7 | 20.0 |
| 9D | 1.8 | 1.6 | 88.0 |
| 9D-Inf | 118.2 | 119.5 | 0.0 |
| 10D | 1.6 | 1.5 | 91.0 |
| RPMG-Quat | 3.5 | 2.4 | 70.0 |
| RPMG-6D | 15.0 | 2.9 | 55.0 |
| RPMG-9D | **1.3** | **1.2** | **97.5** |
| RPMG-10D | 1.5 | 1.4 | 97.0 |

Table 4. **Self-supervised Instance-Level Rotation Estimation from Point Clouds.** We report mean, median and 3° accuracy of (geodesic) errors after 30K iterations.

## 5.4. Regression on Other Non-Euclidean Manifolds

In addition to SO(3), our method can also be applied for regression on other non-Euclidean manifolds as long as the target manifold meets some conditions: 1) the manifold should support Riemannian optimization. 2) the inverse projection $\pi^{-1}$ should be calculable, although it doesn't need to be mathematically complete. Here we show the experiment of *Sphere manifold* $\mathcal{S}^2$.

**Unit vector regression.** For rotational symmetric categories (e.g., *bottle*), the pose of an object is ambiguous. We'd rather regress a unit vector for each object indicating the *up* direction of it. We use the ModelNet-40 [40] *bottle* point cloud dataset. The network architecture is the same as in Experiment 5.1 except the dimension of output is 3.

L2-loss-w/-norm computes L2 loss between the normalized predictions and the ground truth. L2-loss-w/o-norm computes L2 loss between the raw predictions and the ground truth, similar to $\lambda = 1$ and $\tau = \tau_{gt}$. For MG-3D, PMG-3D and RPMG-3D, We increase $\tau$ from 0.1 to 0.5 since here $\tau_{converge} = 0.5$ (please see Appendix C for the derivation).

The results are shown in Table 5. MG-3D performs on par with L2-loss-w/o-norm, and PMG-3D leads to a large error since the length vanishing problem similar to Figure 3. RPMG-3D outperforms all the baselines and variants.

| Methods | Mean (°)↓ | Med (°)↓ | 1°Acc (%)↑ |
|---|---|---|---|
| L2 loss w/ norm | 8.73 | 2.71 | 0.0 |
| L2 loss w/o norm | 5.71 | 1.10 | 37.4 |
| MG-3D ($\lambda$=1) | 5.37 | 1.20 | 22.2 |
| PMG-3D ($\lambda$=0) | 21.96 | 14.79 | 0.0 |
| RPMG-3D ($\lambda$=0.01) | **4.69** | **0.76** | **72.7** |

Table 5. **Unit vector estimation from ModelNet bottle point clouds.** We report mean, median, and 1° accuracy of (geodesic) errors after 30K iterations.

## 6. Conclusion and Future Work

Our work tackles the problem of designing a gradient layer to facilitate the learning of rotation regression. Our extensive experiments have demonstrated the effectiveness of our method coupled with different rotation representations in diverse tasks dealing with rotation estimation.

The limitation of our methods mainly lies in two fronts: 1) we introduce two new hyperparameters, *i.e.*, $\tau$ and $\lambda$, though our performance is not sensitive to them, as long as they are in a reasonable range; 2) as discussed in Sec 5.4, our method can only be applied to manifolds with certain constraints. We leave how to relax those to future works.

## References


[1] P-A Absil, Robert Mahony, and Rodolphe Sepulchre. *Optimization algorithms on matrix manifolds*. Princeton University Press, 2009. 1, 11

[2] Henry Adams, M. Aminian, Elin Farnell, M. Kirby, C. Peterson, Joshua Mirth, R. Neville, P. Shipman, and C. Shonkwiler. A fractal dimension for measures via persistent homology. *arXiv: Dynamical Systems*, pages 1–31, 2020. 5

[3] Tolga Birdal and Umut Simsekli. Probabilistic permutation synchronization using the riemannian structure of the birkhoff polytope. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11105–11116, 2019. 2

[4] Tolga Birdal, Umut Simsekli, Mustafa Onur Eken, and Slobodan Ilic. Bayesian pose graph optimization via bingham distributions and tempered geodesic mcmc. *Advances in Neural Information Processing Systems*, 31, 2018. 2

[5] Jose-Luis Blanco. A tutorial on se (3) transformation parameterizations and on-manifold optimization. *University of Malaga, Tech. Rep*, 3:6, 2010. 1

[6] Nicolas Boumal. An introduction to optimization on smooth manifolds. *Available online, May*, 2020. 3

[7] Romain Brégier. Deep regression on manifolds: a 3d rotation case study. *CoRR*, abs/2103.16317, 2021. 2

[8] Mai Bui, Tolga Birdal, Haowen Deng, Shadi Albarqouni, Leonidas Guibas, Slobodan Ilic, and Nassir Navab. 6d camera relocalization in ambiguous scenes via continuous multimodal inference. *arXiv preprint arXiv:2004.04807*, 2020. 1

[9] Haiwei Chen, Shichen Liu, Weikai Chen, Hao Li, and Randall Hill. Equivariant point network for 3d point cloud analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14514–14523, 2021. 6

[10] Earl A Coddington and Norman Levinson. *Theory of ordinary differential equations*. Tata McGraw-Hill Education, 1955. 11

[11] Haowen Deng, Mai Bui, Nassir Navab, Leonidas Guibas, Slobodan Ilic, and Tolga Birdal. Deep bingham networks: Dealing with uncertainty and ambiguity in pose estimation. *arXiv preprint arXiv:2012.11002*, 2020. 2

[12] Thanh-Toan Do, Ming Cai, Trung Pham, and Ian D. Reid. Deep-6dpose: Recovering 6d object pose from a single RGB image. *CoRR*, abs/1802.10367, 2018. 2

[13] Siyan Dong, Qingnan Fan, He Wang, Ji Shi, Li Yi, Thomas Funkhouser, Baoquan Chen, and Leonidas Guibas. Robust neural routing through space partitions for camera relocalization in dynamic indoor environments. *arXiv preprint arXiv:2012.04746*, 2020. 1

[14] Haoqiang Fan, Hao Su, and Leonidas J Guibas. A point set generation network for 3d object reconstruction from a single image. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 605–613, 2017. 2, 5

[15] Ge Gao, Mikko Lauri, Jianwei Zhang, and Simone Frintrop. Occlusion resistant object rotation regression from point cloud segments. In *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, September 2018. 2

[16] Zan Gojcic, Caifa Zhou, Jan D Wegner, Leonidas J Guibas, and Tolga Birdal. Learning multiview 3d point cloud registration. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1759–1769, 2020. 1

[17] Benjamin Hou, Nina Miolane, Bishesh Khanal, Matthew CH Lee, Amir Alansary, Steven McDonagh, Jo V Hajnal, Daniel Rueckert, Ben Glocker, and Bernhard Kainz. Computing cnn loss and gradients for pose estimation with riemannian geometry. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 756–764. Springer, 2018. 2

[18] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *CoRR*, abs/1704.04861, 2017. 7

[19] Jiahui Huang, He Wang, Tolga Birdal, Minhyuk Sung, Federica Arrigoni, Shi-Min Hu, and Leonidas J Guibas. Multibodysync: Multi-body segmentation and motion estimation via 3d scan synchronization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7108–7118, 2021. 2

[20] Du Q Huynh. Metrics for 3d rotations: Comparison and analysis. *Journal of Mathematical Imaging and Vision*, 35(2):155–164, 2009. 4

[21] Alex Kendall and Roberto Cipolla. Geometric loss functions for camera pose regression with deep learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017. 2

[22] Alex Kendall, Matthew Grimes, and Roberto Cipolla. Posenet: A convolutional network for real-time 6-dof camera relocalization. In *Proceedings of the IEEE international conference on computer vision*, pages 2938–2946, 2015. 1

[23] Alex Kendall, Matthew Grimes, and Roberto Cipolla. Posenet: A convolutional network for real-time 6-dof camera relocalization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015. 2, 16

[24] Abhijit Kundu, Yin Li, and James M. Rehg. 3d-rcnn: Instance-level 3d object reconstruction via render-and-compare. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018. 2

[25] Jake Levinson, Carlos Esteves, Kefan Chen, Noah Snavely, Angjoo Kanazawa, Afshin Rostamizadeh, and Ameesh Makadia. An analysis of svd for deep rotation estimation. *arXiv preprint arXiv:2006.14616*, 2020. 1, 2, 3, 4, 6, 7, 14, 16

[26] Shuai Liao, Efstratios Gavves, and Cees G. M. Snoek. Spherical regression: Learning viewpoints, surface normals and 3d rotations on n-spheres. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, Long Beach, USA, June 2019. 7

[27] Valentin Peretroukhin, Matthew Giamou, David M. Rosen, W. Nicholas Greene, Nicholas Roy, and Jonathan Kelly. A

Smooth Representation of SO(3) for Deep Rotation Learning with Uncertainty. In *Proceedings of Robotics: Science and Systems (RSS'20)*, Jul. 12–16 2020. 1, 2, 3, 5, 6, 12, 14

[28] Charles R Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *arXiv preprint arXiv:1706.02413*, 2017. 6, 16

[29] Olinde Rodrigues. Des lois géométriques qui régissent les déplacements d'un système solide dans l'espace, et de la variation des coordonnées provenant de ces déplacements considérés indépendamment des causes qui peuvent les produire. *Journal de mathématiques pures et appliquées*, 5(1):380–440, 1840. 11

[30] Hao Su, Charles R. Qi, Yangyan Li, and Leonidas J. Guibas. Render for cnn: Viewpoint estimation in images using cnns trained with rendered 3d model views. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015. 2, 14, 15

[31] Camillo J Taylor and David J Kriegman. Minimization on the lie group so (3) and related manifolds. *Yale University*, 16(155):6, 1994. 1, 2, 11

[32] Zachary Teed and Jia Deng. Tangent space backpropagation for 3d transformation groups. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2

[33] Shubham Tulsiani and Jitendra Malik. Viewpoints and keypoints. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015. 2

[34] Constantin Udriste. *Convex functions and optimization methods on Riemannian manifolds*, volume 297. Springer Science & Business Media, 2013. 1

[35] Benjamin Ummenhofer, Huizhong Zhou, Jonas Uhrig, Nikolaus Mayer, Eddy Ilg, Alexey Dosovitskiy, and Thomas Brox. Demon: Depth and motion network for learning monocular stereo. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017. 2

[36] Florian Walch, Caner Hazirbas, Laura Leal-Taixé, Torsten Sattler, Sebastian Hilsenbeck, and Daniel Cremers. Image-based localization using lstms for structured feature correlation. In *ICCV*, October 2017. 15

[37] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. Densefusion: 6d object pose estimation by iterative dense fusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3343–3352, 2019. 1

[38] Gu Wang, Fabian Manhardt, Jianzhun Shao, Xiangyang Ji, Nassir Navab, and Federico Tombari. Self6d: Self-supervised monocular 6d object pose estimation. In *The European Conference on Computer Vision (ECCV)*, August 2020. 4

[39] He Wang, Srinath Sridhar, Jingwei Huang, Julien Valentin, Shuran Song, and Leonidas J Guibas. Normalized object coordinate space for category-level 6d object pose and size estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2642–2651, 2019. 2, 5, 6

[40] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Computer Vision and Pattern Recognition*, 2015. 6, 7, 8, 16

[41] Yu Xiang, Roozbeh Mottaghi, and Silvio Savarese. Beyond pascal: A benchmark for 3d object detection in the wild. In *IEEE Winter Conference on Applications of Computer Vision*, pages 75–82, 2014. 14

[42] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199*, 2017. 2, 4

[43] Li Yi, Haibin Huang, Difan Liu, Evangelos Kalogerakis, Hao Su, and Leonidas Guibas. Deep part induction from articulated object pairs. *ACM Transactions on Graphics*, 37(6), 2019. 2

[44] Hongyi Zhang, Sashank J Reddi, and Suvrit Sra. Riemannian svrg: Fast stochastic optimization on riemannian manifolds. *arXiv preprint arXiv:1605.07147*, 2016. 1

[45] Yongheng Zhao, Tolga Birdal, Jan Eric Lenssen, Emanuele Menegatti, Leonidas Guibas, and Federico Tombari. Quaternion equivariant capsule networks for 3d point clouds. In *European Conference on Computer Vision*, pages 1–19. Springer, 2020. 2

[46] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5745–5753, 2019. 1, 2, 3, 4, 6, 16

## A. More on Riemannian Geometry

In this part, we supplement the definitions in Section 3 to allow for a slightly more rigorous specification of the exponential map for interested readers.

We denote the union of all tangent spaces as the *tangent bundle*: $\mathcal{TM} = \cup_{\mathbf{x}\in\mathcal{M}}\mathcal{T}_{\mathbf{x}}\mathcal{M}$. Riemannian metric $\mathbf{G}_{\mathbf{x}}$ induces a norm $\|\mathbf{u}\|_{\mathbf{x}}, \forall \mathbf{u} \in \mathcal{T}_{\mathbf{x}}\mathcal{M}$ locally defining the geometry of the manifold and allows for computing the *length* of any curve $\gamma : [0,1] \to \mathcal{M}$, with $\gamma(0) = \mathbf{x}$ and $\gamma(1) = \mathbf{y}$ as the integral of its speed: $\ell(\gamma) = \int_0^1 \|\dot{\gamma}(t)\|_{\gamma(t)} dt$. The notion of length leads to a natural notion of distance by taking the infimum over all lengths of such curves, giving the *Riemannian distance* on $\mathcal{M}$, $d(\mathbf{x},\mathbf{y}) = \inf_\gamma \ell(\gamma)$. The constant speed *length minimizing* curve $\gamma$ is called a *geodesic* on $\mathcal{M}$.

By the celebrated Picard Lindelöf theorem [10], given any $(\mathbf{x},\mathbf{v}) \in \mathcal{TM}$, there exists a unique *maximal* geodesic $\gamma_{\mathbf{v}}$ such that $\gamma_{\mathbf{v}}(0) = \mathbf{x}$ and $\dot{\gamma}_{\mathbf{v}}(0) = \mathbf{v}$. Hence, we can define a unique diffeomorphism or *exponential map*, sending $\mathbf{x}$ to the endpoint of the geodesic: $\exp_{\mathbf{x}}(\mathbf{v}) = \gamma_{\mathbf{v}}(1)$. We will refer to the well-defined, smooth inverse of this map as the *logaritmic map*: $\log_{\mathbf{x}} \mathbf{y} \triangleq \exp_{\mathbf{x}}^{-1}(\mathbf{v})$. Note that the geodesic is not the only way to move away from $\mathbf{x}$ in the direction of $\mathbf{v}$ on $\mathcal{M}$. In fact, any continuously differentiable, smooth map $R_{\mathbf{x}} : \mathcal{T}_{\mathbf{x}}\mathcal{M} \mapsto \mathcal{M}$ whose directional derivative along $\mathbf{v}$ is identity, *i.e.* $\mathrm{D}R_{\mathbf{x}}(\mathbf{0})[\mathbf{v}] = \mathbf{v}$ and $R_{\mathbf{x}}(\mathbf{0}) = \mathbf{x}$ allows for moving on the manifold in a given direction $\mathbf{v}$. Such $R_{\mathbf{x}}$, called *retraction*, constitutes the basic building block of any on-manifold optimizer as we use in the main paper. In addition to those we also speak of a *manifold projector* $\pi : \mathcal{X} \mapsto \mathcal{M}$ is available for the manifolds we consider in this paper. Note that, most of these definitions directly generalize to matrix manifolds such as Stiefel or Grassmann [1].

## B. Projective Manifold Gradient on $SO(3)$

### B.1. Details of Riemannian Optimization on $SO(3)$

**Riemannian gradient on $SO(3)$.** Since we mainly focus on the SO(3) manifold in this paper, we will further show the specific expression of some related concepts of SO(3) below.

Firstly, SO(3) is defined as a matrix subgroup of the general linear group $GL(3)$:

$$\mathrm{SO}(3) = \{\mathbf{R} \in \mathbb{R}^{3\times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\}. \quad (7)$$

The tangent space of a rotation matrix in SO(3) is isomorphic to $\mathbb{R}^3$ making SO(3) an embedded submanifold of the ambient Eucldiean space $\mathcal{X}$. Hence, SO(3) *inherits* the metric or the inner product of its embedding space, $\mathcal{X}$.

*maximal* refers to the fact that the curve is as long as possible.

Since SO(3) is also a Lie group, elements of the tangent space $\boldsymbol{\phi}^\wedge \in \mathcal{T}_{\mathbf{I}}\mathcal{M}$ can be uniquely mapped to the manifold $\mathcal{M}$ through the exponential map:

$$\exp_{\mathbf{I}}(\boldsymbol{\phi}^\wedge) = \mathbf{I} + \boldsymbol{\phi}^\wedge + \frac{1}{2!}(\boldsymbol{\phi}^\wedge)^2 + \frac{1}{3!}(\boldsymbol{\phi}^\wedge)^3 + \ldots \quad , \quad (8)$$

where $\mathbf{I} \in \mathrm{SO}(3)$ is the identity matrix and $^\wedge$ is a skew-symmetric operator $^\wedge : \mathbb{R}^3 \to \mathcal{T}_{\mathbf{I}}\mathcal{M}$ as

$$\boldsymbol{\phi}^\wedge = \begin{pmatrix} 0 & -\phi_z & \phi_y \\ \phi_z & 0 & -\phi_x \\ -\phi_y & \phi_x & 0 \end{pmatrix} \quad (9)$$

Due to the nature of the Lie group, we can expand the formula in Eq. (8) from the tangent space of the identity, $\mathcal{T}_{\mathbf{I}}\mathcal{M}$, to $\mathcal{T}_{\mathbf{R}}\mathcal{M}$ by simply multiplying by an $\mathbf{R}$:

$$\exp_{\mathbf{R}}(\boldsymbol{\phi}^\wedge) = \mathbf{R}\left(\sum_{n=0}^{\infty}(\frac{1}{n!}(\boldsymbol{\phi}^\wedge)^n)\right) \quad (10)$$

If the vector $\boldsymbol{\phi}$ is rewritten in terms of a unit vector $\boldsymbol{\omega}$ and a magnitude $\theta$, the exponential map can further be simplified as

$$\mathrm{Exp}_{\mathbf{R}}(\boldsymbol{\phi}) = \mathbf{R}(\mathbf{I} + \sin\theta\, \boldsymbol{\omega}^\wedge + (1-\cos\theta)(\boldsymbol{\omega}^\wedge)^2) \quad (11)$$

which is well known as the Rodrigues formula [29]. Following [31], we have

$$\frac{\partial}{\partial \phi_x}\mathrm{Exp}_{\mathbf{R}}(\boldsymbol{\phi})\Big|_{\boldsymbol{\phi}=\mathbf{0}} = \mathbf{R}\left(\cos\theta\, \frac{\partial\theta}{\partial\phi_x}\boldsymbol{\omega}^\wedge\right)\Big|_{\boldsymbol{\phi}=\mathbf{0}} = \mathbf{R}\mathbf{x}^\wedge \quad (12)$$

where $\mathbf{x} = (1,0,0) \in \mathbb{R}^3$. For $\phi_y$ and $\phi_z$, there are the similar expressions of the gradient. Finally we can have

$$\mathrm{grad}\, \mathcal{L}f(\mathbf{R}) = \left(\frac{\partial f(\mathbf{R})}{\partial \mathbf{R}}\frac{\partial}{\partial\boldsymbol{\phi}}\mathrm{Exp}_{\mathbf{R}}(\boldsymbol{\phi})\Big|_{\boldsymbol{\phi}=\mathbf{0}}\right)^\wedge \quad (13)$$

**Riemannian gradient descent on SO(3).** We are now ready to state the Riemannian optimization in the main paper in terms of the exponential map:

$$\mathbf{R}_{k+1} = \mathrm{Exp}_{\mathbf{R}_k}(-\tau_k \nabla\boldsymbol{\phi}). \quad (14)$$

Note that if we consider the most commonly used L2 loss $f(\mathbf{R}) = \|\mathbf{R} - \mathbf{R}_{\mathrm{gt}}\|_F^2$ , where

$$\mathbf{R} = \begin{pmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{pmatrix} \in \mathrm{SO}(3), \quad \mathbf{R}_{\mathrm{gt}} = \begin{pmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{pmatrix} \in \mathrm{SO}(3),$$

we can get an analytical expression of $\nabla\boldsymbol{\phi} = (\nabla\phi_x, \nabla\phi_y, \nabla\phi_z)$ as follows:

$$\begin{aligned} \nabla\boldsymbol{\phi}_x &= \frac{\partial f(\mathbf{R})}{\partial\mathbf{R}} * \mathbf{R}\mathbf{x}^\wedge \\ &= 2\left\|\begin{pmatrix} a_1-x_1 & b_1-y_1 & c_1-z_1 \\ a_2-x_2 & b_2-y_2 & c_2-z_2 \\ a_3-x_3 & b_3-y_3 & c_3-z_3 \end{pmatrix}\begin{pmatrix} 0 & c_1 & -b_1 \\ 0 & c_2 & -b_2 \\ 0 & c_3 & -b_3 \end{pmatrix}\right\|_1 \\ &= 2 * \sum_{i=1}^{3}(b_i * z_i - c_i * y_i) \end{aligned} \quad (15)$$

Similarly, we have $\nabla\phi_y = 2 * \sum_{i=1}^{3}(c_i * x_i - a_i * z_i)$ and $\nabla\phi_z = 2 * \sum_{i=1}^{3}(a_i * y_i - b_i * x_i)$.

**$\tau_{converge}$ in ablation study.** We have mentioned in Section 4.3 that $\tau$ should be small at the beginning of training and be large when converging. This is because a small $\tau$ can yield $\mathbf{R}_g$ closer to $\mathbf{R}$ and greatly alleviate the reverse problem at the beginning stage of training discussed in Section 4.3. Later in training, a large $\tau$ can help us converge better. The initial $\tau$ will not influence the final results too much, and we just need to choose a reasonable value. But the final $\tau$ matters.

Right before convergence, our ideal choice for the final $\tau$ would be $\tau_{\text{gt}}$. Given that the value of $\tau_{\text{gt}}$ will change according to the geodesic distance between $\mathbf{R}$ and $\mathbf{R}_{\text{gt}}$, we instead choose to find a suitable constant value to act like $\tau_{\text{gt}}$ when converging, which we denotes as $\tau_{converge}$.

**Lemma 1.** *The final value of $\tau_{converge}$ satisfies:*

$$\mathbf{R}_{\text{gt}} = \lim_{<\mathbf{R},\mathbf{R}_{\text{gt}}>\to 0} R_{\mathbf{R}}(-\tau_{converge} \text{ grad } \mathcal{L}(f(\mathbf{R}))) \tag{16}$$

*where $< \mathbf{R}, \mathbf{R}_{\text{gt}} >$ represents the angle between $\mathbf{R}$ and $\mathbf{R}_{\text{gt}}$.*

*Proof.* Considering the symmetry, without loss of generality, we assume that $\mathbf{R} = \mathbf{I}$, which will simplify the derivation. Based upon the conclusion in Eq. (15), when we use L2 loss, we have $\nabla\boldsymbol{\phi} = (2 * (z_2 - y_3), 2 * (x_3 - z_1), 2 * (y_1 - x_2))$ and grad $\mathcal{L}f(\mathbf{R}) = (\nabla\boldsymbol{\phi})^\wedge = 2(\mathbf{R}_{\text{gt}}^\top - \mathbf{R}_{\text{gt}})$. Taking the manifold logarithm of both sides, we get:

$$\log_{\mathbf{R}}(\mathbf{R}_{\text{gt}}) = \lim_{<\mathbf{R},\mathbf{R}_{\text{gt}}>\to 0} -\tau_{converge} \text{ grad } \mathcal{L}f(\mathbf{R}) \tag{17}$$

The solution for $\tau_{converge}$ can then be derived as follows:

$$\begin{aligned}
\tau_{converge} &= \lim_{<\mathbf{R},\mathbf{R}_{\text{gt}}>\to 0} -\frac{\log_{\mathbf{R}}(\mathbf{R}_{\text{gt}})}{\text{grad } \mathcal{L}f(\mathbf{R})} \\
&= \lim_{\theta\to 0} -\frac{(\boldsymbol{\phi}_{\text{gt}})^\wedge}{2(\mathbf{R}_{\text{gt}}^\top - \mathbf{R}_{\text{gt}})} \\
&= \lim_{\theta\to 0} -\frac{(\boldsymbol{\phi}_{\text{gt}})^\wedge}{2\sin\theta(((\boldsymbol{\omega}_{\text{gt}})^\wedge)^\top - (\boldsymbol{\omega}_{\text{gt}})^\wedge)} \\
&= \lim_{\theta\to 0} \frac{\theta}{4\sin\theta} \\
&= \frac{1}{4}
\end{aligned} \tag{18}$$

where $(\boldsymbol{\phi}_{\text{gt}})^\wedge = \log_{\mathbf{I}}(\mathbf{R}_{\text{gt}}) = \theta(\boldsymbol{\omega}_{\text{gt}})^\wedge, \theta =< \mathbf{I}, \mathbf{R}_{\text{gt}} >$ □

Note that though $\tau_{converge} = \frac{1}{4}$ is only true for the L2 loss, we can solve $\tau_{converge}$ for other frequently used loss formats, *e.g.*, geodesic loss [27]. If we use geodesic loss $\theta^2$, it can be computed that $\tau_{converge} = \frac{1}{2}$. We leave the detailed derivation to the interested readers.

## B.2. Derivations of Inverse Projection

For different rotation representations, we follow the same process to find its inverse projection: we first find the inverse image space $\pi^{-1}(\mathbf{x}_g)$, then project $\mathbf{x}$ to this space resulting in $\mathbf{x}_{gp}$, and finally get our (regularized) projective manifold gradient.

**Quaternion** We need to solve

$$\mathbf{x}_{gp} = \underset{\mathbf{x}_g \in \pi_q^{-1}(\hat{\mathbf{x}}_g)}{\text{argmin}} \|\mathbf{x}_g - \mathbf{x}\|_2^2, \tag{19}$$

where $\mathbf{x}$ is the raw output of our network in *ambient space* $\mathbb{R}^4$, $\hat{\mathbf{x}}_g$ is the next goal in *representation manifold* $\mathcal{S}^3$, and $\mathbf{x}_g$ is the variable to optimize in *ambient space* $\mathbb{R}^4$. Recall $\pi_q^{-1}(\hat{\mathbf{x}}_g) = \{\mathbf{x} \mid \mathbf{x} = k\hat{\mathbf{x}}_g, k \in \mathbb{R} \text{ and } k > 0\}$, and we can have

$$\|\mathbf{x} - \mathbf{x}_g\|_2^2 = \mathbf{x}^2 - 2k\mathbf{x}\cdot\hat{\mathbf{x}}_g + k^2\hat{\mathbf{x}}_g^2 \tag{20}$$

Without considering the condition of $k > 0$, We can see when $k = \frac{\mathbf{x}\cdot\hat{\mathbf{x}}_g}{\hat{\mathbf{x}}_g^2} = \mathbf{x}\cdot\hat{\mathbf{x}}_g$ the target formula reaches minimum. Note that when using a small $\tau$, the angle between $\hat{\mathbf{x}}_g$ and $\mathbf{x}$ is always very small, which means the condition of $k = \mathbf{x}\cdot\hat{\mathbf{x}}_g > 0$ can be satisfied naturally. For the sake of simplicity and consistency of gradient, we ignore the limitation of $k$ no matter what value $\tau$ takes. Therefore, the inverse projection is $\mathbf{x}_{gp} = (\mathbf{x}\cdot\hat{\mathbf{x}}_g)\hat{\mathbf{x}}_g$.

**6D representation** We need to solve

$$[\mathbf{u}_{gp}, \mathbf{v}_{gp}] = \underset{[\mathbf{u}_g,\mathbf{v}_g]\in\pi_{6D}^{-1}([\hat{\mathbf{u}}_g,\hat{\mathbf{v}}_g])}{\text{argmin}} (\|\mathbf{u}_g - \mathbf{u}\|_2^2 + \|\mathbf{v}_g - \mathbf{v}\|_2^2) \tag{21}$$

where $[\mathbf{u}, \mathbf{v}]$ is the raw output of network in *ambient space* $\mathbb{R}^6$, $[\hat{\mathbf{u}}_g, \hat{\mathbf{v}}_g]$ is the next goal in *representation manifold* $\mathcal{V}_2(\mathbb{R}^3)$ and $[\mathbf{u}_g, \mathbf{v}_g]$ is the variable to optimize in *ambient space* $\mathbb{R}^6$. Recall $\pi_{6D}^{-1}([\hat{\mathbf{u}}_g, \hat{\mathbf{v}}_g]) = \{[k_1\hat{\mathbf{u}}_g, k_2\hat{\mathbf{u}}_g + k_3\hat{\mathbf{v}}_g] \mid k_1, k_2, k_3 \in \mathbb{R} \text{ and } k_1, k_3 > 0\}$. We can see that $\mathbf{u}_g$ and $\mathbf{v}_g$ are independent, and $\mathbf{u}_g$ is similar to the situation of quaternion. So we only need to consider the part of $\mathbf{v}_g$ as below:

$$\|\mathbf{v} - \mathbf{v}_g\|_2^2 = \mathbf{v}^2 + k_2^2\hat{\mathbf{u}}_g^2 + k_3^2\hat{\mathbf{v}}_g^2 - 2k_2\mathbf{v}\cdot\hat{\mathbf{u}}_g - 2k_3\mathbf{v}\cdot\hat{\mathbf{v}}_g \tag{22}$$

For the similar reason as quaternion, we ignore the condition of $k_3 > 0$ and we can see when $k_2 = \mathbf{v}\cdot\hat{\mathbf{u}}_g$ and $k_3 = \mathbf{v}\cdot\hat{\mathbf{v}}_g$, the target formula reaches minimum. Therefore, the inverse projection is $[\mathbf{u}_{gp}, \mathbf{v}_{gp}] = [(\mathbf{u}\cdot\hat{\mathbf{u}}_g)\hat{\mathbf{u}}_g, (\mathbf{v}\cdot\hat{\mathbf{u}}_g)\hat{\mathbf{u}}_g + (\mathbf{v}\cdot\hat{\mathbf{v}}_g)\hat{\mathbf{v}}_g]$

**9D representation** For this representation, obtaining the inverse image $\pi_{9D}^{-1}$ is not so obvious. Recall $\pi_{9D}(\mathbf{x}) = \mathbf{U}\Sigma'\mathbf{V}^\top$, where $\mathbf{U}$ and $\mathbf{V}$ are left and right singular vectors of $\mathbf{x}$ decomposed by SVD expressed as $\mathbf{x} = \mathbf{U}\Sigma\mathbf{V}^\top$, and $\Sigma' = \text{diag}(1, 1, \det(\mathbf{U}\mathbf{V}^\top))$.

**Lemma 2.** *The inverse image* $\pi_{9D}^{-1}(\mathbf{R}_g) = \{\mathbf{S}\mathbf{R}_g \mid \mathbf{S} = \mathbf{S}^\top\}$ *satisfies that* $\{\mathbf{x}_g \mid \pi_{9D}(\mathbf{x}_g) = \mathbf{R}_g\} \subset \pi_{9D}^{-1}(\mathbf{R}_g)$.

*Proof.* To find a suitable $\pi_{9D}^{-1}$, the most straightforward way is to only change the singular values $\Sigma_g = \mathrm{diag}(\lambda_0, \lambda_1, \lambda_2)$, where $\lambda_0, \lambda_1, \lambda_2$ can be arbitrary scalars, and recompose the $\mathbf{x}_g = \mathbf{U}\Sigma_g\mathbf{V}^\top$.

However, we argue that this simple method will fail to capture the entire set of $\{\mathbf{x}_g \mid \pi_{9D}(\mathbf{x}_g) = \mathbf{R}_g\}$, because different $\mathbf{U}'$ and $\mathbf{V}'$ can yield the same rotation $\mathbf{R}_g$. In fact, $\mathbf{U}_g$ can be arbitrary if $\mathbf{x}_g = \mathbf{U}_g\Sigma_g\mathbf{V}_g^\top$ and $\mathbf{U}_g\Sigma_g'\mathbf{V}_g^\top = \mathbf{R}_g$. Assuming $\mathbf{R}_g$ is known, we can replace $\mathbf{V}_g^\top$ by $\mathbf{R}_g$ and express $\mathbf{x}_g$ in a different way: $\mathbf{x}_g = \mathbf{U}_g\Sigma_g\frac{1}{\Sigma_g'}\mathbf{U}_g^{-1}\mathbf{R}_g$. Notice that $\mathbf{U}_g\Sigma_g\frac{1}{\Sigma_g'}\mathbf{U}_g^{-1}$ must be a symmetry matrix since $\mathbf{U}_g$ is an orthogonal matrix. Therefore, $\{\mathbf{x}_g \mid \pi_{9D}(\mathbf{x}_g) = \mathbf{R}_g\} \subseteq \pi_{9D}^{-1}(\mathbf{R}_g) = \{\mathbf{S}\mathbf{R}_g \mid \mathbf{S} = \mathbf{S}^\top\}$.

Note that such $\mathbf{x}_g \in \pi_{9D}^{-1}(\mathbf{R}_g)$ can't ensure $\pi_{9D}(\mathbf{x}_g) = \mathbf{R}_g$, because in the implementation of SVD, the order and the sign of three singular values are constrained, which is not taken into consideration. Therefore, $\{\mathbf{x}_g \mid \pi_{9D}(\mathbf{x}_g) = \mathbf{R}_g\} \neq \pi_{9D}^{-1}(\mathbf{R}_g)$. □

Then we need to solve

$$\mathbf{x}_{gp} = \underset{\mathbf{x}_g \in \pi_{9D}^{-1}(\mathbf{R}_g)}{\mathrm{argmin}} \|\mathbf{x}_g - \mathbf{x}\|_2^2 \tag{23}$$

where $\mathbf{x}$ is the raw output of our network in *ambient space* $\mathbb{R}^{3\times3}$, $\hat{\mathbf{x}}_g$ is the next goal in *representation manifold* $\mathrm{SO}(3)$, and $\mathbf{x}_g$ is the variable to optimize in *ambient space* $\mathbb{R}^{3\times3}$. We can further transform the objective function as below:

$$\|\mathbf{x}_g - \mathbf{x}\|_2^2 = \|\mathbf{S}\mathbf{R}_g - \mathbf{x}\|_2^2 = \|\mathbf{S} - \mathbf{x}\mathbf{R}_g^\top\|_2^2 \tag{24}$$

Now we can easily find when $\mathbf{S}$ equals to the symmetry part of $\mathbf{x}\mathbf{R}_g^\top$, the target formula reaches minimum. Therefore, the inverse projection admits a simple form $\mathbf{x}_{gp} = \frac{\mathbf{x}\mathbf{R}_g^\top + \mathbf{R}_g\mathbf{x}^\top}{2}\mathbf{R}_g$.

**10D representation** Recall the *manifold mapping* $\pi_{10D} : \mathbb{R}^{10} \to \mathcal{S}^3, \pi_{10D}(\mathbf{x}) = \min_{\mathbf{q}\in\mathcal{S}^3} \mathbf{q}^\top\mathbf{A}(\mathbf{x})\mathbf{q}$, in which

$$\mathbf{A}(\boldsymbol{\theta}) = \begin{pmatrix} \theta_1 & \theta_2 & \theta_3 & \theta_4 \\ \theta_2 & \theta_5 & \theta_6 & \theta_7 \\ \theta_3 & \theta_6 & \theta_8 & \theta_9 \\ \theta_4 & \theta_7 & \theta_9 & \theta_{10} \end{pmatrix}. \tag{25}$$

We need to solve

$$\mathbf{x}_{gp} = \underset{\mathbf{A}(\mathbf{x}_g)\mathbf{q}_g = \lambda\mathbf{q}_g}{\arg\min} \|\mathbf{x}_g - \mathbf{x}\|_2^2, \tag{26}$$

where $\mathbf{x}$ is the raw output of our network in *ambient space* $\mathbb{R}^{10}$, $\mathbf{q}_g$ is the next goal in *representation manifold* $\mathcal{S}^3$, and $\mathbf{x}_g$ is the variable to optimize in *ambient space* $\mathbb{R}^{10}$. Note that $\lambda$ is also a variable to optimize. For the similar reason as before, for the sake of simplicity and consistency of analytical solution, here we also need to relax the constraint that $\lambda$ should be the smallest eigenvalue of $\mathbf{A}(\mathbf{x}_g)$.

To solve Eq. 25, we start from rewriting $\mathbf{A}(\mathbf{x}_g)\mathbf{q}_g = \lambda\mathbf{q}_g$ as

$$\mathbf{M}\Delta\mathbf{x} = \lambda\mathbf{q}_g - \mathbf{A}(\mathbf{x})\mathbf{q}_g, \tag{27}$$

where $\Delta\mathbf{x} = \mathbf{x}_g - \mathbf{x}$ and

$$\mathbf{M} = \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & q_1 & 0 & 0 & q_2 & q_3 & q_4 & 0 & 0 & 0 \\ 0 & 0 & q_1 & 0 & 0 & q_2 & 0 & q_3 & q_4 & 0 \\ 0 & 0 & 0 & q_1 & 0 & 0 & q_2 & 0 & q_3 & q_4 \end{pmatrix} \tag{28}$$

where $\mathbf{q}_g = (q_1, q_2, q_3, q_4)^\top$. For simplicity, we denote $\lambda\mathbf{q}_g - \mathbf{A}(\mathbf{x})\mathbf{q}_g$ as $\mathbf{b}$.

Once we have finished the above steps for preparation, we solve $\lambda$ and $\Delta\mathbf{x}$ for the minimal problem by two steps as below. First, we assume $\lambda$ is known and the problem becomes that given $\mathbf{M}$ and $\mathbf{b}$, we need to find the best $\Delta\mathbf{x}$ to minimize $\|\Delta\mathbf{x}\|_2^2$ with the constraint $\mathbf{M}\Delta\mathbf{x} = \mathbf{b}$. This is a typical quadratic optimization problem with linear equality constraints, and the analytical solution satisfies

$$\begin{pmatrix} \mathbf{I} & \mathbf{M}^\top \\ \mathbf{M} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \mathbf{v} \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ \mathbf{b} \end{pmatrix} \tag{29}$$

where $\mathbf{v}$ is a set of Lagrange multipliers which come out of the solution alongside $\Delta\mathbf{x}$, and $\begin{pmatrix} \mathbf{I} & \mathbf{M}^\top \\ \mathbf{M} & \mathbf{0} \end{pmatrix}$ is called KKT matrix. Since this matrix has full rank almost everywhere, we can multiple the inverse of this KKT matrix in both sides of Eq. 29 and lead to the solution of $\Delta\mathbf{x}$ as below:

$$\begin{pmatrix} \Delta\mathbf{x} \\ \mathbf{v} \end{pmatrix} = \begin{pmatrix} \mathbf{I} & \mathbf{M}^\top \\ \mathbf{M} & \mathbf{0} \end{pmatrix}^{-1} \begin{pmatrix} \mathbf{0} \\ \mathbf{b} \end{pmatrix} \tag{30}$$

Recall that $\mathbf{b} = \lambda\mathbf{q}_g - \mathbf{A}(\mathbf{x})\mathbf{q}_g$, therefore so far we have had the solution of $\Delta\mathbf{x}$ respect to each $\lambda$:

$$\Delta\mathbf{x} = \begin{pmatrix} \Delta\mathbf{x} \\ \mathbf{v} \end{pmatrix}_{0:10} = \mathbf{K}(\lambda\mathbf{q}_g - \mathbf{A}(\mathbf{x})\mathbf{q}_g) = \lambda\mathbf{S} - \mathbf{T} \tag{31}$$

in which $\mathbf{K}$ is the upper right part of the inverse of the KKT matrix $\mathbf{K} = \left[\begin{pmatrix} \mathbf{I} & \mathbf{M}^\top \\ \mathbf{M} & \mathbf{0} \end{pmatrix}^{-1}\right]_{10:14,0:10}$, $\mathbf{S} = \mathbf{K}\mathbf{q}_g$ and $\mathbf{T} = \mathbf{K}\mathbf{A}(\mathbf{x})\mathbf{q}_g$.

Next, we need to optimize $\lambda$ to minimize our objective function $\|\Delta\mathbf{x}\|_2^2$. In fact, using the results of Eq. 31, $\|\Delta\mathbf{x}\|_2^2$ becomes a quadratic functions on $\lambda$, thus we can simply get the final analytical solution of $\lambda$ and $\mathbf{x}_{gp}$:

$$\begin{cases} \lambda = \frac{(\mathbf{S}^\top\mathbf{T} + \mathbf{T}^\top\mathbf{S})}{2\mathbf{S}^\top\mathbf{S}} \\ \mathbf{x}_{gp} = \mathbf{x} + \lambda\mathbf{S} - \mathbf{T} \end{cases} \tag{32}$$

Another thing worth mentioning here is that in this special case, the *representation manifold* $\mathcal{S}^3$ is no longer a subspace of the *abmient space* $\mathbb{R}^{10}$, which means that we can't directly compute our regularization term $\mathbf{x}_{gp} - \mathbf{q}_g$ because $\mathbf{x}_{gp} \in \mathbb{R}^{10}$ while $\mathbf{q}_g \in \mathcal{S}^3$. However, the length vanishing problem still exists as shown in Figure 3. Therefore, to compute the regularization term, we need a simple mapping to convert $\mathbf{q}_g$ to an element on $\mathbb{R}^{10}$ with stable length norm. We use the mapping $g : \mathcal{S}^3 \to \mathbb{R}^{10}, g(\mathbf{q}) = \mathbf{A}^{-1}(\mathbf{I} - \mathbf{q}\mathbf{q}^\top)$, which is proposed in [27]. They also proved that $\pi(g(\mathbf{q})) = \mathbf{q}$ is always true, which makes $g(\mathbf{q})$ better than simply normalizing $\mathbf{x}_{gp}$ because the latter one will suffer from the problem of opposite gradient discussed in Section 4.3.

# C. Projective Manifold Gradient on $S^2$

## C.1. Riemannian Optimization on $S^2$

Our methods can also be applied for the regression of other manifolds. Taking $\mathcal{S}^2$ as an example, which is included in Experiment 5.4, we will show the detail of how our projective manifold gradient layer works in other manifolds.

During forward, The network predicts a raw output $\mathbf{x} \in \mathbb{R}^3$, which is then mapped to $\hat{\mathbf{x}} \in \mathcal{S}^2$ through a *manifold mapping* $\pi(\mathbf{x}) = \mathbf{x}/\|\mathbf{x}\|$. Here we don't define the *rotation mapping* and *representation mapping*, and we directly compute the loss function on *representation manifold* $\mathcal{S}^2$.

During backward, to apply a Riemannian optimization, we first need to know some basic concepts of $\mathcal{S}^2$. The tangent space of an arbitrary element $\hat{\mathbf{x}} \in \mathcal{S}^2$ is $\mathcal{T}_{\hat{\mathbf{x}}}\mathcal{M}$, which is a plane. And we can map a geodesic path $\mathbf{v} \in \mathcal{T}_{\hat{\mathbf{x}}}\mathcal{M}$ to an element on the manifold $\mathcal{S}^2$ through $\exp_{\hat{\mathbf{x}}}(\mathbf{v}) = \cos(\|\mathbf{v}\|)\hat{\mathbf{x}} + \sin(\|\mathbf{v}\|)\frac{\mathbf{v}}{\|\mathbf{v}\|}$, where $\|.\|$ means the ordinal Frobenius norm.

For the definition of the mapping $^\wedge$, which connects Euclidean space $\mathbb{R}^2$ and the tangent space $\mathcal{T}_{\hat{\mathbf{x}}}\mathcal{M}$, we need to first define two orthogonal axes $\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2$ in the tangent plane. Note that the choice of $\hat{\mathbf{c}}_1$ and $\hat{\mathbf{c}}_2$ won't influence the final result, which will be shown soon after. To simplify the derivation, we can assume ground truth unit vector $\hat{\mathbf{x}}_{gt}$ is known and choose $\hat{\mathbf{c}}_1 = \frac{\mathrm{Log}_{\hat{\mathbf{x}}}(\hat{\mathbf{x}}_{gt})}{\|\mathrm{Log}_{\hat{\mathbf{x}}}(\hat{\mathbf{x}}_{gt})\|} = \frac{\hat{\mathbf{x}}_{gt} - (\hat{\mathbf{x}}_{gt} \cdot \hat{\mathbf{x}})}{\|\hat{\mathbf{x}}_{gt} - (\hat{\mathbf{x}}_{gt} \cdot \hat{\mathbf{x}})\|}$ and $\hat{\mathbf{c}}_2 = \hat{\mathbf{x}} \times \hat{\mathbf{c}}_1$. Then we can say $\boldsymbol{\phi}^\wedge = \phi_1\hat{\mathbf{c}}_1 + \phi_2\hat{\mathbf{c}}_2$, where $\boldsymbol{\phi} = (\phi_1, \phi_2) \in \mathbb{R}^2$. The gradient of exponential mapping with respect to $\boldsymbol{\phi}$ is

$$\begin{aligned}
&\frac{\partial}{\partial \phi_1}\mathrm{Exp}_{\hat{\mathbf{x}}}(\boldsymbol{\phi})\Big|_{\boldsymbol{\phi}=\mathbf{0}} \\
&= \frac{\partial}{\partial \phi_1}(\cos(\|\phi_1\hat{\mathbf{c}}_1\|)\hat{\mathbf{x}} + \sin(\|\phi_1\hat{\mathbf{c}}_1\|)\frac{\phi_1\hat{\mathbf{c}}_1}{\|\phi_1\hat{\mathbf{c}}_1\|})\Big|_{\boldsymbol{\phi}=\mathbf{0}} \\
&= \hat{\mathbf{c}}_1
\end{aligned} \tag{33}$$

Similarly, we have $\frac{\partial}{\partial \phi_2}\mathrm{Exp}_{\hat{\mathbf{x}}}(\boldsymbol{\phi})\Big|_{\boldsymbol{\phi}=\mathbf{0}} = \hat{\mathbf{c}}_2$.

When using L2 loss, we can have

$$\begin{aligned}
\mathrm{grad}\ \mathcal{L}f(\hat{\mathbf{x}}) &= (\nabla f(\hat{\mathbf{x}}))^\wedge = (\nabla \boldsymbol{\phi})^\wedge \\
&= \left(\frac{\partial f(\hat{\mathbf{x}})}{\partial \hat{\mathbf{x}}}\frac{\partial}{\partial \boldsymbol{\phi}}\mathrm{Exp}_{\hat{\mathbf{x}}}(\boldsymbol{\phi})\Big|_{\boldsymbol{\phi}=\mathbf{0}}\right)^\wedge \\
&= ((2(\hat{\mathbf{x}} - \hat{\mathbf{x}}_{gt})\hat{\mathbf{c}}_1, 2(\hat{\mathbf{x}} - \hat{\mathbf{x}}_{gt})\hat{\mathbf{c}}_2))^\wedge \\
&= 2((\hat{\mathbf{x}} \cdot \hat{\mathbf{x}}_{gt})\hat{\mathbf{x}} - \hat{\mathbf{x}}_{gt})
\end{aligned} \tag{34}$$

Note that this expression doesn't depend on the choice of $\hat{\mathbf{c}}_1$ and $\hat{\mathbf{c}}_2$.

Similar to Eq 18, we can also solve a $\tau_{converge}$

$$\tau_{converge} = \lim_{<\hat{\mathbf{x}}, \hat{\mathbf{x}}_{\mathrm{gt}}> \to 0} -\frac{\mathrm{Log}_{\hat{\mathbf{x}}}(\hat{\mathbf{x}}_{\mathrm{gt}})}{\mathrm{grad}\ \mathcal{L}f(\hat{\mathbf{x}})} = \lim_{\theta \to 0}\frac{\theta\hat{\mathbf{c}}_1}{2\sin\theta\hat{\mathbf{c}}_1} = \frac{1}{2} \tag{35}$$

where $\theta =< \hat{\mathbf{x}}, \hat{\mathbf{x}}_{gt} >$. Note that in Experiment 5.4, we change the schedule of $\tau$ according to this conclusion. We increase $\tau$ from 0.1 to 0.5 by uniform steps.

## C.2. Inverse Projection

Similar to quaternion, we can have $\mathbf{x}_{gp} = (\mathbf{x} \cdot \hat{\mathbf{x}}_g)\hat{\mathbf{x}}_g$. For the detail of derivation, see Section B.2.

# D. Computational Cost

Our method does not alter the forward pass and thus incurs no cost at test time. For backward pass at training, we observe that, before and after inserting RMPG layers, the backward time for quaternion / 6D / 9D / 10D representations, averaged among 1K iterations on a GeForce RTX 3090, changes from 4.39 / 4.48 / 4.48 / 4.53 to 4.45 / 4.43 / 4.49 / 4.63 (unit: $10^{-2}$ s), and the memory cost changes from 11449 / 11415 / 10781 / 11363 to 11457 / 11459 / 11545 / 11447 (unit: MiB). Note that the runtime is almost keep the same, as Riemannian optimization only performs an additional projection and we derive and always use analytical solutions in representation mapping, inversion and projection steps. RPMG also has a very marginal cost on the memory, as it does not introduce any weights but only a few intermediate variables.

# E. More Experiments

## E.1. Pascal3D+

Pascal3D+ [41] is a standard benchmark for object pose estimation from real images. We follow the same setting as in [25] to estimate object poses from single images. For training we discard occluded or truncated objects and augment with rendered images from [30]. In the Table 6 and Table 7, we report our results on *sofa* and *bicycle* categories. We use the same batch size as in [25]. As for the learning rate, we use the same strategy as in Experiment 5.2. See the discussion in Section F.2.

It can be seen that our method leads to consistent improvements to quaternion, 6D, 9D and 10D representations on both *sofa* and *bicycle* classes. One may be curious about

| Methods | Accuracy(%) ↑ | | | Med(°) ↓ |
|---|---|---|---|---|
| | 10° | 15° | 20° | Err |
| Euler | 60.2 | 80.9 | 90.6 | 8.3 |
| Axis-Angle | 45.0 | 70.9 | 85.1 | 11.0 |
| Quaternion | 34.3 | 60.8 | 73.5 | 13.2 |
| 6D | 50.8 | 76.7 | 89.0 | 9.9 |
| 9D | 52.4 | 79.6 | 90.3 | 9.2 |
| 9D-Inf | 70.9 | **88.0** | 93.5 | **6.7** |
| 10D | 50.2 | 77.0 | 89.6 | 9.8 |
| RPMG-Quat | 56.6 | 79.6 | 90.9 | 8.9 |
| RPMG-6D | 69.6 | 86.1 | 92.2 | **6.7** |
| RPMG-9D | **72.5** | **88.0** | **95.8** | **6.7** |
| RPMG-10D | 69.3 | 87.1 | 93.9 | 7.0 |

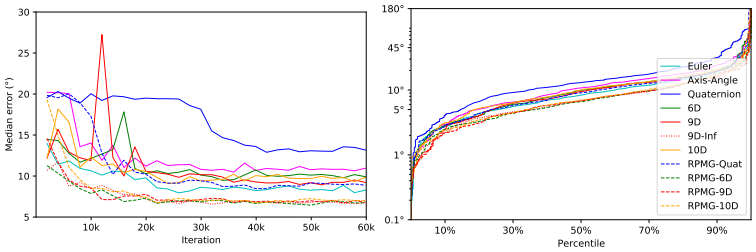


Table 6. **Pose estimation from PASCAL3D+ *sofa* images.** Left: a comparison of methods by 10° / 15° / 20° accuracy of (geodesic) errors and median errors after 60k training steps. Middle: median test error at different iterations during training. Right: test error percentiles after training completes. The legend on the right applies to both plots.

| Methods | Accuracy(%) ↑ | | | Med(°) ↓ |
|---|---|---|---|---|
| | 10° | 15° | 20° | Err |
| Euler | 28.2 | 48.1 | 62.7 | 15.7 |
| Axis-Angle | 5.3 | 8.1 | 10.1 | 79.7 |
| Quaternion | 20.8 | 38.8 | 54.6 | 18.7 |
| 6D | 21.8 | 39.0 | 55.3 | 18.1 |
| 9D | 20.6 | 37.6 | 56.9 | 18.0 |
| 9D-Inf | 38.0 | 53.3 | 69.9 | 13.4 |
| 10D | 23.9 | 42.3 | 56.7 | 17.9 |
| RPMG-Quat | 32.3 | 50.0 | 65.6 | 15.0 |
| RPMG-6D | 35.4 | 57.2 | 70.6 | 13.5 |
| RPMG-9D | 36.8 | 57.4 | **71.8** | **12.5** |
| RPMG-10D | **40.0** | **57.7** | 71.3 | 12.9 |

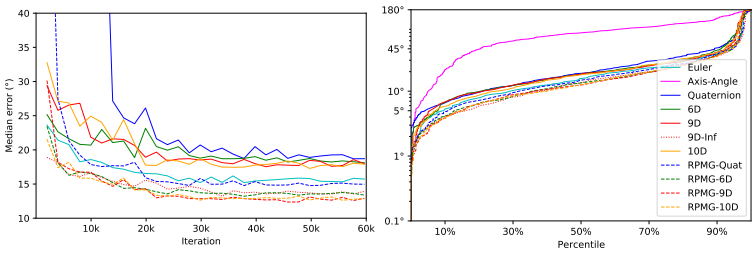


Table 7. **Pose estimation from PASCAL3D+ *bicycle* images.** We report the same metrics as Table 6; see the caption there.

why our method can only outperform 9D-inf for a margin. We think that this is because this dataset is quite challenging. The number of annotated real image for training is only around 200 for each category. Though there are a lot of synthetic images generated from [30] for training, these images suffer from sim-to-real domain gap. Therefore, we argue that the bottleneck here is not in optimization, which makes the gains from less noise in gradient smaller(Note that 9D-inf is just a special case of our methods with $\lambda = 1$ and $\tau = \tau_{gt}$). But compared to vanilla 4D/6D/9D/10D representation, our methods can still bring a great improvement.

## E.2. Using Flow Loss for Rotation Estimation from Point Clouds.

Apart from the most widely used L2 loss, our method can also be applied to the loss of other forms, e.g. flow loss.

We mainly follow the setting in Experiment 5.1 with *airplane* point clouds dataset and the only difference is that we use flow loss $\|\mathbf{R}X - \mathbf{R}_{gt}X\|_F^2$ here, where $X$ is the complete point clouds.

Since the format of loss is changed, the previous schedule of $\tau$ is not suitable anymore, and we have to change the value of $\tau$ accordingly. Our selection skill is to first choose a $\tau$ as we like and visualize the mean geodesic distance between predicted $\mathbf{R}$ and $\mathbf{R}_g$ during training. Then we can roughly adjust $\tau$ to make the geodesic distance looked reasonable. For this experiment, we use $\tau = 50$ and $\lambda = 0.01$. In Table 8, we show our methods again outperform vanilla methods as well as **9D-inf**.

| Methods | Mean (°) | Med (°) | 5°Acc (%) |
|---|---|---|---|
| Euler | 12.14 | 6.91 | 33.6 |
| Axis-Angle | 35.49 | 20.80 | 4.7 |
| Quaternion | 11.54 | 7.67 | 29.8 |
| 6D | 14.13 | 9.41 | 23.4 |
| 9D | 11.44 | 8.01 | 23.8 |
| 9D-Inf | 4.07 | 3.28 | 76.7 |
| 10D | 9.28 | 7.05 | 32.6 |
| RPMG-Quat | 4.86 | 3.25 | 75.8 |
| RPMG-6D | **2.71** | **2.04** | **92.1** |
| RPMG-9D | 3.75 | 2.10 | 91.1 |
| RPMG-10D | 3.30 | 2.70 | 86.8 |

Table 8. **Flow Loss for Rotation Estimation from Point Clouds.** All models are trained for 30K iterations.

## E.3. Camera Relocalization

The task of camera relocalization is to estimate a 6 Degree-of-Freedom camera pose (rotation and translation) from visual observations, which is a fundamental component of many computer vision and robotic applications. In this experiment, we use all the settings (data, network, training strategy, hyperparameters, etc.) of PoseLSTM [36] except that we modify the rotation representations and the

| Methods | King's College | | Old Hospital | | Shop Facade | | St Mary's Church | | Average | |
|---|---|---|---|---|---|---|---|---|---|---|
| | T($m$) | R(°) | T($m$) | R(°) | T($m$) | R(°) | T($m$) | R(°) | T($m$) | R(°) |
| Euler | 1.16 | 2.85 | 2.54 | 2.95 | 1.25 | 6.48 | 1.98 | 6.97 | 1.73 | 4.81 |
| Axis-Angle | 1.12 | 2.63 | 2.41 | 3.38 | **0.84** | 5.05 | 2.16 | 7.58 | 1.63 | 4.66 |
| Quaternion | **0.98** | 2.50 | 2.39 | 3.44 | 1.06 | 6.01 | 2.59 | 8.81 | 1.76 | 5.19 |
| 6D | 1.10 | 2.56 | 2.21 | 3.43 | 1.01 | 5.43 | **1.73** | 5.82 | 1.51 | 4.31 |
| 9D | 1.14 | 3.03 | 2.11 | 3.50 | 0.88 | 6.39 | 1.95 | 5.95 | 1.52 | 4.72 |
| 9D-Inf | **0.98** | 2.32 | **1.89** | 3.32 | 1.15 | 6.36 | 1.96 | 6.25 | **1.50** | 4.56 |
| 10D | 1.54 | 2.62 | 2.32 | 3.39 | 1.20 | 5.76 | 1.85 | 6.69 | 1.73 | 4.62 |
| RPMG-Quat | 1.04 | 1.91 | 2.42 | 2.72 | 0.98 | 4.28 | 1.82 | 4.89 | 1.57 | 3.45 |
| RPMG-6D | 1.55 | **1.70** | 2.62 | 3.09 | 0.95 | 5.01 | 2.44 | 5.18 | 1.89 | 3.75 |
| RPMG-9D | 1.57 | 1.82 | 4.37 | 3.12 | 0.93 | 4.17 | 1.92 | **4.69** | 2.20 | 3.45 |
| RPMG-10D | 1.30 | 1.74 | 3.21 | **2.59** | 1.10 | **3.47** | 2.20 | 5.09 | 1.95 | **3.22** |

Table 9. **Camera relocalization on Cambridge Landscape dataset.** We report the *median* error of translation and rotation of the best checkpoint, which is chosen by minimizing the median of rotation. We only care about the rotation error here.

gradient layers. We report the results on the outdoor Cambridge Landscape dataset [23] in Table 9.

Notice that our RPMG layer performs the best on the rotation regression task, but not on the translation regression. We believe this results from a loss imbalance. We does not change the weights of the rotation loss and translation loss, otherwise it leads to an unfair comparison with existing results. We only care about the rotation error here.

# F. More Implementation Details

## F.1. Experiment 5.1 & 5.3 & 5.4

**Data** We generate the data from ModelNet dataset [40] by sampling 1024 points on the mesh surface, following the same generation method as in [46]. We uniformly sample M rotations for each data point and set them as the ground truth. We apply the sampled rotations on the canonical point clouds to obtain the input data.

**Network Architecture** We use a PointNet++ MSG [28] backbone as our feature extractor. Our network takes input a point cloud with a resolution of 1024. It them performs three set abstractions to lower the resolution to 512, 128, and finally 1, resulting in a global feature of dimensionality 1024. The feature is finally pushed through a three-layer MLP $[1024, 512, N]$ to regress rotation, where $N$ is the dimension of the rotation representation.

**Training details** The learning rate is set to 1e-3 and decayed by 0.7 every 3k iterations. The batch size is 20. For each experiment, we train the network on one NVIDIA TITAN Xp GPU for 30k iterations.

## F.2. Experiment 5.2

Most of the training settings and strategies are all the same as [25] except learning rate. We find setting initial learning rate $lr = 1e-3$ and decaying to $1e-5$ can perform much better than using $lr = 1e-5$ as in [25], which accounts for the inconsistency of the results of those baseline methods compared to [25]. We believe that the methods should be compared under hyperparameters as optimal as possible. Thus, we stick to our $lr$ schedule.

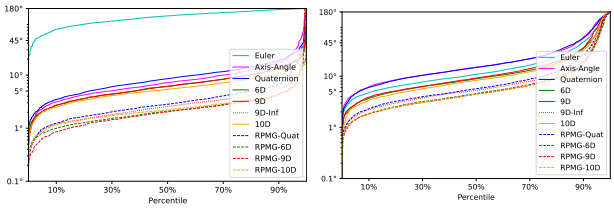


Table 10. **Test error percentiles for Experiment 5.1 & 5.2** Left: test error percentiles of *airplane* for Experiment 5.1 after training completes. Right: test error percentiles of *chair* for Experiment 5.2 after training completes.

# G. Addition on Rotation Representations

**Standard mapping between rotation matrix and unit quaternion** The *rotation mapping* $\phi : \mathbf{q} \mapsto \mathbf{R}$ algebraically manipulates a unit quaternion $\mathbf{q}$ into a rotation matrix:

$$\phi(\mathbf{q}) = \begin{pmatrix} 2(q_0^2+q_1^2)-1 & 2(q_1q_2-q_0q_3) & 2(q_1q_3+q_0q_2) \\ 2(q_1q_2+q_0q_3) & 2(q_0^2+q_2^2)-1 & 2(q_2q_3-q_0q_1) \\ 2(q_1q_3-q_0q_2) & 2(q_2q_3+q_0q_1) & 2(q_0^2+q_3^2)-1 \end{pmatrix} \quad (36)$$

where $\mathbf{q} = (q_0, q_1, q_2, q_3) \in \mathcal{S}^3$.

In the reverse direction, the *representation mapping* $\psi(\mathbf{R})$ can be expressed as:

$$\begin{cases} q_0 = \sqrt{1+R_{00}+R_{11}+R_{22}}/2 \\ q_1 = (R21 - R_{12})/(4*q_0) \\ q_2 = (R_{02} - R_{20})/(4*q_0) \\ q_3 = (R_{10} - R_{01})/(4*q_0) \end{cases} \quad (37)$$

Note that $\mathbf{q} = (q_0, q_1, q_2, q_3)$ and $-\mathbf{q} = (-q_0, -q_1, -q_2, -q_3)$ both are the valid quaternions parameterizing the same $\mathbf{R}$.